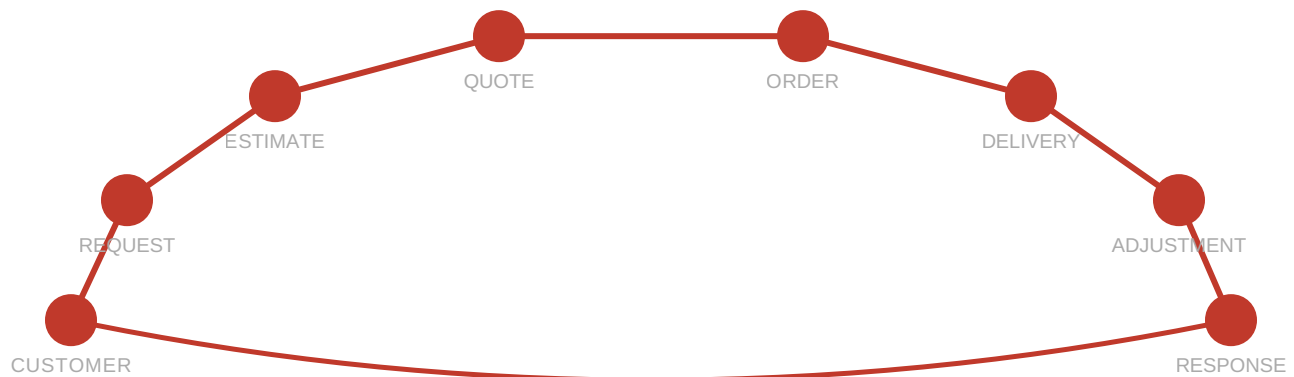




The THREAD

Customer Sales Data: A Practical Guide to Building and Using Sales Intelligence

From a single customer record to a fully operational **sales intelligence system** built across ten chapters, four tiers, and every step of the customer journey.

PHASE I · TEXTBOOK

Tier I · II · III · IV

Table of Contents

FRONT MATTER

- **Introduction: The Customer's Data Journey** Tier I
Preface · The universal customer journey · Tier I schema reference · Baseline KPIs · How to use this textbook
-

TIER I — FOUNDATION

- 01 Knowing Your Customer** I → II
Contact data · Customer segmentation · Account hierarchy · Why a name is not enough
-
- 02 Capturing the Request** I → II
Request types · Product vs. service workflows · Request metadata · The origination record
-

TIER II — LINE-LEVEL DETAIL

- 03 Estimating with Confidence** II
Cost vs. price modeling · Margin tracking · SKU introduction · Estimate lines
-
- 04 Quoting for Clarity** II
Quote versioning · Approval workflows · Expiry tracking · Quote-to-order linkage
-
- 05 Managing the Order** II → III
Order lines · Quantities · Scheduling · Partial fulfillment · Status lifecycle
-

TIER III — OPERATIONAL DEPTH

- 06 Delivering on the Promise** III
Delivery tracking · Actual vs. promised dates · Carrier data · Days variance
-
- 07 When Things Change** III
Adjustment types · Root cause analysis · Financial impact · Credit workflows
-
- 08 The Customer's Voice** III
Response scoring · NPS integration · Sentiment · Follow-up queue · Silent customers
-

TIER IV — INTELLIGENCE LAYER

- 09 Measuring Performance** III → IV
KPI design · Period tables · Dashboard logic · Period-over-period analysis
-

PREFACE

This textbook is designed for practitioners — the people who work inside sales operations, customer success, order management, and business intelligence teams. It does not assume you are a data scientist. It assumes you work with customers, handle transactions, and at some point have asked: *"Why can't I just get a straight answer from our data?"*

The answer to that question usually lies not in the tools, but in the foundation. Data that lacks structure, discipline, and a clear model of the customer journey will frustrate every analyst, every dashboard, and every business review it touches. This textbook builds that foundation — from a single customer record to a fully operational sales intelligence system.

INTRODUCTION

The Customer's Data Journey

When managing information about the services and products you offer, a strong understanding of the customer's journey from a data perspective is essential. Debris comes in the form of redundant information, personal bias, and reckless KPI hunts. Communication gets lost. Teams build shadow spreadsheets. Reports contradict each other. All of this stems from one root cause: the data model was never designed around the actual journey the customer takes.

CORE CONCEPT

The Universal Customer Journey

Across industries and organizations, the most common supporting data follows the same logical progression — not a sales funnel, but a **data lineage**. Each step creates a record that references the step before it.

- 1 **Customer** — *Who is asking?*
- 2 **Customer Request** — *What are they asking for?*
- 3 **Estimate** — *What will it take to fulfill the request?*
- 4 **Quote** — *What is the formal offer?*
- 5 **Order** — *What has been agreed upon?*

6 Delivery — *What was actually delivered, and when?*

7 Adjustments — *What changed along the way?*

8 Customer Response — *How did the customer react?*

This chain of references is what allows you to trace any outcome back to its origin: a delivery dispute back to the original quote, a profitability variance back to the estimate, a customer churn signal back to unresolved adjustments.

FOUNDATION

Six Baseline Questions

The Tier I schema is intentionally minimal — a clean, unambiguous foundation before complexity is added. It answers six baseline questions that remain the health check throughout the entire textbook:

? How many open orders exist?

? How many customers do we have?

? What is our delivery rate?

? How is our customer satisfaction?

? Who are our most profitable customers?

? Who are our new customers?

The Eight Foundation Tables

The following tables constitute the Tier I foundation. Each defines a step in the journey with its primary key, foreign keys linking it to prior steps, and the minimum attributes needed to support the six baseline questions.

Column	Attribute	Notes
CUSTOMER		
cust_id	Customer ID	PK — anchor of the entire data model
cust_type	Customer Type	e.g., Direct, Reseller, Internal
cust_name	Customer Name	Legal or preferred name
cust_branch	Customer Branch	Division, location, or business unit
REQUEST		
req_id	Request ID	PK
cust_id	Customer ID	FK → customer
req_type	Request Type	e.g., New, Renewal, Change
cust_req	Customer Request	Free-text or structured description
req_delivery_date	Requested Delivery Date	Customer's stated target; origin point for all downstream scheduling comparisons
ESTIMATE		
est_id	Estimate ID	PK
req_id	Request ID	FK → request
cust_id	Customer ID	FK → customer
est_shipment_date	Estimated Delivery Date	Internal working estimate
est_cost	Estimated Cost	Internal cost to fulfill
est_price	Estimated Price	Price to present to customer
est_alerts	Estimate Alerts	Flags for risk, capacity, or dependency issues
QUOTE		
qte_id	Quote ID	PK
est_id	Estimate ID	FK → estimate
req_id / cust_id		FK → request · customer

	Request ID / Customer ID	
qte_shipment_date	Quoted Delivery Date	Committed date in the offer
qte_cost / qte_price	Quoted Cost / Price	Cost basis and price offered
ORDER		
ord_id	Order ID	PK
qte_id / est_id / req_id / cust_id	Full lineage FKs	Each order knows its complete origin chain
ord_exp_date	Expected Delivery Date	Operational target date (split into two fields in Ch5)
ord_cost / ord_price	Order Cost / Price	Confirmed cost and price
DELIVERY		
del_id	Delivery ID	PK
ord_id ... cust_id	Full lineage FKs	FK chain to order, quote, estimate, request, customer
del_date	Delivery Date	Actual delivery date
ADJUSTMENT		
adj_id	Adjustment ID	PK
del_id ... cust_id	Full lineage FKs	FK chain to delivery and all prior steps
adj_type	Adjustment Type	e.g., Price, Date, Quantity, Credit
adj_reason	Adjustment Reason	Explanation of why the change occurred
adj_del / adj_cost / adj_price	Date / Cost / Price Adjustment	The change values
RESPONSE		
resp_id	Customer Response ID	PK — closing record of the cycle
del_id ... cust_id	Full lineage FKs	FK chain to delivery and all prior steps
cust_response	Customer Response	Feedback, satisfaction score, or follow-up request

Foreign Key Redundancy — Intentional

Several tables carry the full chain of foreign keys — `cust_id`, `req_id`, `est_id`, etc. — even when they are technically derivable by joining through parent tables. This is an intentional Tier I design choice for two reasons:

1. **Query simplicity** — analysts can join directly to any record without traversing multiple tables
2. **Audit transparency** — every record knows its complete lineage without requiring a join

As the schema matures in later tiers, some redundancy is deliberately normalized out. Chapter 4 renames `est_id` on the quote table to `qte_est_id` to align naming conventions. Chapter 5 replaces `ord_exp_date` with two purpose-specific date fields. Each change is documented with a supersession note in the relevant chapter. For now, clarity wins over purity.

HOW TO USE THIS TEXTBOOK

Chapter Structure

Each chapter follows a consistent five-part structure:

1. **Business Context** — The real-world situation motivating the new data need
2. **The Data Gap** — What the current schema cannot answer
3. **Schema Update** — New columns or tables, with rationale for every addition
4. **Analytical Application** — SQL queries, example reports, or KPI definitions
5. **Reflection** — Questions to apply the concept in your own organization

The schema workbook (`Customer_Processing_Schema.xlsx`) is updated alongside each chapter. Each worksheet maps to a table, and the **Tier** column tracks when each attribute was introduced. You do not need to build everything at once — the tier system is designed for incremental implementation.

Before You Begin

Look at the Tier I schema reference. List three questions you believe it can already support. Then list three questions you suspect it cannot yet answer. The second list will become the roadmap for the chapters ahead.



CHAPTER 1

Knowing Your Customer

Tier I → II · Contact data, customer segmentation, account hierarchy

Beyond the Four Fields

The Tier I `customer` table gives you four things: an ID, a type, a name, and a branch. That is enough to count customers, link them to transactions, and answer basic branch-level questions. It is not enough to actually *know* them.

Consider what happens when a delivery goes wrong. You need to call someone. Who? The `customer` table has no phone number, no email, no name of the person who placed the order. You know the company and the branch. You do not know the contact.

Or consider a quarterly business review. Your VP asks which customer segments are driving the most revenue. You have `cust_type`, but it was entered inconsistently — some records say "Direct," others say "direct client," others are blank. The field exists, but the data is not usable.

This is the pattern that Tier II addresses: the foundation is in place, but the real world demands more precision, more structure, and more people.

THE DATA GAP

What You Can't Answer Yet

With only the Tier I `customer` table, here are the questions you **cannot** reliably answer:

- Who is the primary contact at this account?
- Who should receive the quote? Who approves the order?
- How many distinct branches are associated with a customer — and are they all being served consistently?
- Is this customer a subsidiary of a larger parent account we also serve?
- What industry is this customer in, and how does that affect how we serve them?

A note on branches: `cust_branch` in Tier I allows you to ask *how many branches are associated with a customer* — but only as reliably as that field has been populated. It is a starting point, not a structure. Tier II formalizes the relationship so the answer is authoritative rather than approximate.

Tier II: New Tables

```
-- contact: the person behind the account
contact_id      INT PK
customer_id     INT FK → customer
contact_name    VARCHAR
contact_role    VARCHAR -- e.g. Buyer, Approver, Receiver
contact_email   VARCHAR
contact_phone   VARCHAR

-- customer_segment: replaces inconsistent cust_type free text
segment_id      INT PK
customer_id     INT FK → customer
segment_type    VARCHAR -- controlled vocabulary
industry        VARCHAR
parent_account  INT FK → customer -- nullable
```

⚠ WATCH OUT

Don't collapse contacts into the customer record. It's tempting to add a `contact_name` column directly to `customer` — but one customer has many contacts. The moment you do this, you've locked yourself into a one-contact model that breaks on the first multi-stakeholder account.

KEY CONCEPT

A customer is an organization. A contact is a person. These are fundamentally different records and should never be collapsed. Contacts change; customers don't. Personnel turns over — when a contact leaves, you update their record, not the customer.

"The foundation is in place — but the real world demands more precision, more structure, and more people. Tier II is where the data starts to reflect how business actually works."



CHAPTER 2

Capturing the Request

Tier I → II · Request types, product vs. service workflows, request metadata

The Beginning of Everything

The request is the beginning of everything. Before there is an estimate, a quote, an order, or a delivery, there is a customer who asked for something. How well you capture that ask determines how cleanly the rest of the data chain flows.

Tier I gives you three fields to describe a request: a type, a free-text description, and the customer it belongs to. That is enough to know a request happened. It is not enough to understand *what kind* of request it was, *when* it came in, *how* it arrived, or *how urgent* the customer considers it.

Think about what happens when a request moves through your organization without that context. An operations team picks up a service request and begins scheduling technicians — only to learn it was actually a product inquiry that should have gone to the warehouse. A high-priority customer submits a request through a web form and it sits in the same queue as routine reorders. A request that came in six weeks ago has no timestamp, so nobody knows whether the follow-up is overdue by days or by months.

These are not edge cases. They are the daily reality of organizations that treat the request as a formality rather than a data event.

THE DATA GAP

What You Can't Answer Yet

- When was this request submitted?
- Is this a product request or a service request — and does it need different routing?
- How did the request reach us — phone, email, portal, in-person?
- How urgent is this request, and does urgency match how we are treating it?
- Which requests are still open, and for how long?
- Who on our side is responsible for following up?
- When does the customer need delivery — and how does that compare to what we ultimately promised?

The Tier I `req_type` field touches on some of these — but type, channel, priority, and status are four distinct dimensions. They belong in four distinct fields.

Tier II: Updated request Table

Column	Attribute	Notes
req_date	Request Date	Timestamp of receipt; system-generated — never manually entered
req_category	Request Category	Product or Service — the fundamental routing split
req_channel	Request Channel	Phone, Email, Portal, In-Person, EDI
req_priority	Request Priority	High, Medium, Low — or SLA tier labels
req_status	Request Status	Open · In Review · Pending Info · Converted · Closed–Fulfilled · Closed–Declined
req_owner	Request Owner	FK → user.user_id; internal user responsible for advancing the request
req_cont_id	Contact ID	FK → contact.cont_id; customer contact who submitted (optional)

Note: `req_delivery_date` was established in Tier I. It is the scheduling anchor for all fields above — no new addition required.

⚠ WATCH OUT

Don't use `req_type` to carry four jobs at once. When a single field encodes category, channel, priority, and status simultaneously, it becomes unqueryable. You end up with values like "High-Priority-Phone-Product" that can't be grouped, filtered, or trended without string parsing. Separate fields enable clean analysis.

KEY CONCEPT

A request that is "Converted" has become an estimate or an order. Its job in the request table is done — the downstream tables carry it forward. A request that has been "Open" for thirty days with no status change is a signal worth investigating. Status is what makes a request list operational rather than historical.

Queries That Become Possible

Open requests by priority

```
SELECT req_priority, COUNT(req_id) AS open_requests
FROM request
WHERE req_status = 'Open'
GROUP BY req_priority
ORDER BY CASE req_priority
  WHEN 'High' THEN 1 WHEN 'Medium' THEN 2 ELSE 3 END;
```

Average age of open requests by channel

```
SELECT req_channel,
  ROUND(AVG(CURRENT_DATE - req_date::DATE), 1) AS avg_days_open
FROM request
WHERE req_status = 'Open'
GROUP BY req_channel
ORDER BY avg_days_open DESC;
```

If portal requests average 4 days open and phone requests average 14, that is an operational signal — not just a curiosity.

Request-to-order conversion rate by category

```
SELECT r.req_category,
  COUNT(DISTINCT r.req_id) AS total_requests,
  COUNT(DISTINCT o.ord_id) AS converted_to_order,
  ROUND(COUNT(DISTINCT o.ord_id)::NUMERIC /
    NULLIF(COUNT(DISTINCT r.req_id), 0) * 100, 1) AS conversion_pct
FROM request r
```

```
LEFT JOIN "order" o ON r.req_id = o.req_id  
GROUP BY r.req_category;
```

Owners with the most requests stalled in review

```
SELECT u.user_first_name || ' ' || u.user_last_name AS owner_name,  
       COUNT(r.req_id) AS stalled_requests,  
       ROUND(AVG(CURRENT_DATE - r.req_date::DATE), 1) AS avg_days_in_review  
FROM request r  
JOIN "user" u ON r.req_owner = u.user_id  
WHERE r.req_status = 'In Review'  
GROUP BY u.user_id, owner_name  
ORDER BY stalled_requests DESC;
```

Questions Worth Sitting With

- *How does your organization currently distinguish between product and service requests? Is that distinction captured in your data, or does it live in someone's head or inbox?*
 - *Look at the last ten requests your team processed. Could you reconstruct when each one arrived, how it arrived, who was responsible for it, and when the customer needed delivery? If not, which field would have made the biggest difference?*
 - *What does your current request-to-order conversion rate look like? If you cannot calculate it, what does that tell you about your request data?*
 - *Think about a request that stalled or fell through the cracks in the past year. Which of the Tier II fields added in this chapter — date, category, channel, priority, status, owner — would have made that request visible before it became a problem?*
 - *Consider the gap between when customers submit requests and when they say they need delivery. Does that gap vary by channel, by customer type, or by product category? What would knowing that gap tell you about your quoting and scheduling capacity?*
-
-



CHAPTER 3

Estimating with Confidence

Tier II · Cost vs. price modeling, margin tracking, SKU introduction, user accountability

Where Ambition Meets Reality

An estimate is your organization's internal answer to a customer's request. It is the moment where you translate *"can you do this?"* into *"here is what it will take, here is what it will cost us, and here is what we will charge."*

Tier I captures the skeleton of that answer: an estimated delivery date, a cost, a price, and a field for alerts. That is enough to know an estimate existed. It is not enough to understand *how* it was built, *what it is made of*, or *how confidently it was made*.

Consider a job that comes in over budget at delivery. You pull up the estimate and find a single cost figure: \$14,200. Was that one line item or forty? Was it a product cost, a labor cost, or both? Did anyone flag a risk before the job started? The data cannot tell you. The estimate was recorded as an outcome, not as a process — and when something goes wrong, there is nothing to audit.

Tier II changes that. It introduces the building blocks of the estimate: individual products and services being priced, the margin being targeted, structured risk flagging, and — for the first time in this schema — the person responsible for the work.

THE DATA GAP

What You Can't Answer Yet

- What specific products or services are included in this estimate?
- What is the margin on this estimate, and is it within acceptable bounds?
- How was the estimated cost derived — catalog price, custom calculation, or a guess?
- Were there risks or constraints flagged during estimation, and what were they?
- How does this estimate compare to similar ones for similar requests?
- Who prepared this estimate, and when?

The Tier I `est_alerts` field acknowledges that alerts exist, but stores them as free text. A flag buried in a text field is not actionable data — it is a note. Tier II makes alerts structured, trackable, and queryable.

Cost vs. Price: The Margin Gap

The Tier I estimate table already carries both `est_cost` and `est_price`. But the presence of both fields creates an obligation: use them correctly and consistently, or the margin data they produce is worthless.

Cost is what it takes to fulfill the request — materials, labor, overhead, logistics. **Price** is what the customer pays: cost plus your margin, adjusted for market conditions, customer tier, and strategy. The gap between them is margin — and margin is one of the most important numbers in any sales operation.

$$\text{Estimated Margin \%} = (\text{est_price} - \text{est_cost}) / \text{est_price} \times 100$$

This requires no new fields — it is derived from what Tier I already provides. At the estimate stage, margin is still a target. It has not been committed to a quote or tested against a customer response. That makes the estimate the earliest and most valuable place to catch structurally unprofitable deals before they ever leave the building.

Introducing SKUs

A SKU — Stock Keeping Unit — is the atomic unit of what you sell. Without SKUs, an estimate for "installation services" is a description. With a SKU, it is a reference to a defined service item with a standard cost, a standard price, a lead time, and a history of how it has been estimated and delivered.

SKUs belong in their own table — a catalog of everything you offer. The estimate references a SKU by ID rather than copying its details. This is the principle of normalization: store information once, reference it everywhere. A single estimate can cover multiple products or services via **estimate lines** — one row per SKU, per estimate.

Estimate Alerts: From Text to Structure

Tier II replaces the free-text `est_alerts` field with two structured fields: `est_alert_type` (a controlled vocabulary — e.g., *Supply Risk*, *Margin Below Threshold*, *Lead Time Uncertainty*, *Requires Approval*) and `est_alert_notes` (free-text detail for human review).

This separation — type for filtering and routing, notes for context — is a pattern that will appear throughout this textbook. Structured fields answer questions. Free-text fields provide explanations. Both are needed; neither can do the other's job.

Individual Accountability and the User Table

Estimates are prepared by people. The naive approach is a text field: `est_prepared_by = 'Maria Santos'` — until she appears as "M. Santos" in one record and "msantos" in another. Text fields applied to people decay into inconsistency. The right approach: store a reference to a `user` table.

The `user` table serves three purposes beyond the estimate: (1) **Role consolidation** — a user's role is stored once and referenced everywhere; (2) **Consistent ownership** — `req_owner` from Ch2 and every future assignment field all reference this same table, enabling cross-table workload queries; (3) **Future security** — in Phase II, this table anchors row-level security and audit logging.

NEW TABLE: `USER`

Column	Attribute	Notes
<code>user_id</code>	User ID	Primary key
<code>user_first_name</code>	First Name	
<code>user_last_name</code>	Last Name	
<code>user_email</code>	Email Address	Unique — used as login identifier
<code>user_role</code>	Role	Estimator, Sales Rep, Operations Manager, Approver, etc.
<code>user_department</code>	Department	Sales, Operations, Finance, etc.
<code>user_active</code>	Active Flag	Boolean — false if the user has left the organization

NEW TABLE: `SKU`

Column	Attribute	Notes
<code>sku_id</code>	SKU ID	Primary key
<code>sku_code</code>	SKU Code	Human-readable identifier (e.g., PROD-1042, SVC-INSTALL-A)
<code>sku_name</code>	SKU Name	Descriptive name of the product or service
<code>sku_category</code>	SKU Category	Product or Service; mirrors <code>req_category</code> from Ch2
<code>sku_unit</code>	Unit of Measure	Each, Hour, Day, Pound, Linear Foot, etc.
<code>sku_std_cost</code>	Standard Cost	Default internal cost; baseline for estimation
<code>sku_std_price</code>	Standard Price	Default list price; baseline for quoting
<code>sku_active</code>	Active Flag	Boolean — false if discontinued or unavailable

NEW TABLE: **ESTIMATE_LINE**

Column	Attribute	Notes
est_line_id	Estimate Line ID	Primary key
est_id	Estimate ID	FK → estimate
sku_id	SKU ID	FK → sku
est_line_qty	Quantity	Number of units of this SKU in the estimate
est_line_cost	Line Cost	Actual cost for this line (may differ from sku_std_cost)
est_line_price	Line Price	Actual price for this line (may differ from sku_std_price)
est_line_notes	Line Notes	Optional override rationale for this line

UPDATED: **ESTIMATE** TABLE

Column	Attribute	Notes
est_date	Estimate Date	Timestamp when estimate was prepared; system-generated
est_prepared_by	Prepared By	FK → user.user_id
est_alert_type	Alert Type	Supply Risk · Margin Below Threshold · Lead Time Uncertainty · Requires Approval
est_alert_notes	Alert Notes	Free-text detail supporting the alert type; supersedes Tier I est_alerts

Also: req_owner from Ch2 is now formally FK → user.user_id with the user table established here.

Queries That Become Possible

Estimated margin on each open estimate

```
SELECT e.est_id, c.cust_name, e.est_cost, e.est_price,
       ROUND((e.est_price - e.est_cost) / NULLIF(e.est_price, 0) * 100, 1) AS margin_pct
FROM estimate e
JOIN customer c ON e.cust_id = c.cust_id
JOIN request r ON e.req_id = r.req_id
WHERE r.req_status NOT IN ('Closed - Fulfilled', 'Closed - Declined')
ORDER BY margin_pct ASC;
```

Sorting ascending surfaces the most at-risk deals first — the ones most likely to be unprofitable if they convert.

Estimates with active alerts requiring approval

```
SELECT e.est_id, c.cust_name,
       u.user_first_name || ' ' || u.user_last_name AS prepared_by,
       e.est_alert_type, e.est_alert_notes, e.est_date
FROM estimate e
JOIN customer c ON e.cust_id = c.cust_id
JOIN "user" u ON e.est_prepared_by = u.user_id
WHERE e.est_alert_type = 'Requires Approval'
ORDER BY e.est_date ASC;
```

Estimated cost vs. standard cost variance by SKU

```
SELECT s.sku_code, s.sku_name,
       ROUND(AVG(el.est_line_cost), 2) AS avg_estimated_cost,
       s.sku_std_cost AS standard_cost,
       ROUND(AVG(el.est_line_cost) - s.sku_std_cost, 2) AS avg_cost_variance
FROM estimate_line el
JOIN sku s ON el.sku_id = s.sku_id
```

```
GROUP BY s.sku_id, s.sku_code, s.sku_name, s.sku_std_cost
ORDER BY avg_cost_variance DESC;
```

A persistent positive variance may mean standard cost is outdated. Negative variance may mean unapproved discounting.

Most accurate estimators (estimate vs. order actuals)

```
SELECT u.user_first_name || ' ' || u.user_last_name AS estimator,
       COUNT(e.est_id) AS estimates_prepared,
       ROUND(AVG(ABS(e.est_cost - o.ord_cost)), 2) AS avg_cost_variance,
       ROUND(AVG(ABS(e.est_price - o.ord_price)), 2) AS avg_price_variance
FROM estimate e
JOIN "user" u ON e.est_prepared_by = u.user_id
JOIN "order" o ON e.est_id = o.est_id
GROUP BY u.user_id, estimator
ORDER BY avg_cost_variance ASC;
```

Users with both open requests and pending estimates

```
SELECT u.user_id, u.user_first_name || ' ' || u.user_last_name AS user_name,
       COUNT(DISTINCT r.req_id) AS open_requests,
       COUNT(DISTINCT e.est_id) AS pending_estimates
FROM "user" u
LEFT JOIN request r ON r.req_owner = u.user_id AND r.req_status = 'Open'
LEFT JOIN estimate e ON e.est_prepared_by = u.user_id AND e.est_alert_type IS NOT
NULL
GROUP BY u.user_id, user_name
HAVING COUNT(DISTINCT r.req_id) > 0 AND COUNT(DISTINCT e.est_id) > 0
ORDER BY open_requests DESC;
```

This cross-table query is only possible because both req_owner and est_prepared_by reference the same user table.

Questions Worth Sitting With

- *When your organization prepares an estimate today, is it a single number or a collection of line items? If it is a single number, what information is lost when that estimate is reviewed months later?*
 - *Think about your most common products or services. Could you assign each one a SKU with a standard cost and a standard price? What would prevent that standardization, and what would it take to get there?*
 - *How are people currently identified in your data systems? Are names stored as text in multiple places? What inconsistencies has that created, and what would a single user record resolve?*
 - *If you could query every open request and every pending estimate by the person responsible for each — in a single query — what would that tell you about your team's current workload distribution?*
 - *Has your organization ever delivered a job that was profitable on paper but a loss in practice? What information, captured at the estimate stage, might have predicted that outcome?*
-



CHAPTER 4

Quoting for Clarity

Tier II · Quote versioning, approval workflows, expiry tracking, data vs. artifacts

The First Artifact the Customer Sees

The estimate was an internal document. The quote is the first artifact the customer sees — and that distinction changes everything. An estimate is typically the result of an internal conversation: a working analysis of cost, feasibility, and timing. A quote is different. In the real world, a quote is almost certainly a shared document: a PDF, a formal letter, an email with attached pricing, or a portal-generated summary. It carries your organization's name, a price, a delivery commitment, and an implied promise. Once sent, it creates expectations. Once accepted, it becomes the contractual foundation of the order.

CORE CONCEPT

Data and Artifacts: The Distinction That Matters

Data is the structured record your organization owns, secures, and controls — it lives in your database, is referenced by other records, and governed by your access policies. **An artifact** is the presentation of that data, formatted for a specific audience. A quote document sent to a customer is an artifact: derived from the data, but not the data itself.

The data outlives the artifact. A quote PDF may be deleted, lost, or never saved. The data record persists, traceable and auditable. This is why the schema must capture everything of operational value — not assume the document will always be available.

The artifact is shaped for its audience. A quote document shows the customer price and delivery date. It does not show your internal cost, your margin targets, or your alert flags. The same underlying data produces different artifacts for different audiences.

Data is owned; artifacts are shared. The estimate existed entirely in the internal sphere. The quote crosses the boundary — and from this point forward, every record has a potential external representation.

THE DATA GAP

What You Can't Answer Yet

- How many revisions did this quote go through before acceptance?
- What changed between quote versions?
- Who approved this quote before it was sent?
- How many active quotes have passed their validity date?
- How long does it typically take customers to respond to a quote?

- Which approver has the highest conversion rate?

Quote Versioning

In practice, quotes are rarely accepted on the first draft. Pricing gets negotiated. Delivery dates shift. Line items get added or removed. In a Tier I schema where each quote is a single mutable record, every revision destroys the history of what came before it.

The solution is a **self-referencing parent key** — the same pattern introduced for account hierarchy in Chapter 1. A new column `qte_parent_id` references `qte_id` in the same table. The original quote has no parent (`NULL`). Each revision points back to the quote it superseded. A companion integer `qte_version` makes the sequence explicit: version 1 is the original, incrementing per revision. The active quote is always the highest version number within a `qte_parent_id` group.

When a deal closes, you can now trace the full revision history — how many versions it took, what changed at each step, and whether the final price was above or below the initial offer. This is the negotiation audit trail that every sales manager wants but almost no one has captured cleanly.

CORE CONCEPT

Revisions vs. Adjustments: The Right Boundary

Quote versioning handles changes made *before the quote is accepted* — negotiation-stage revisions where no order exists yet. The `adjustment` table handles a fundamentally different kind of change: corrections made *after agreement has been reached* — against a committed order or completed delivery.

Mechanism	Timing	Trigger	Recorded In
<code>qte_parent_id</code> / <code>qte_version</code>	Pre-acceptance	Negotiation, scope change, pricing update	quote table — new row per version
<code>adj_*</code>	Post-acceptance	Price correction, date change, quantity change, credit	adjustment table — linked to <code>ord_id</code> or <code>del_id</code>

KEY CONCEPT

A practical test: *"Has the customer said yes yet?"* If no — use a quote revision. If yes — use an adjustment. This separation keeps each table clean and purposeful: the quote table tells the story of how an agreement was reached; the adjustment table tells the story of how reality diverged from it after the fact.

Approval Workflows

Not every quote should leave your organization without a second set of eyes. Three fields capture the complete approval record: `qte_prepared_by` (FK → user), `qte_approved_by` (FK → user; NULL if not yet approved), and `qte_approved_date`. A quote with an approver but no date is a data quality flag — approval happened verbally without documentation. Once consistently populated, you can compare conversion rates by approver: if one manager's quotes convert at 60% and another's at 35%, that is a pricing strategy signal worth investigating.

Quote Expiry

Quotes have a shelf life. Costs change, capacity fills, supplier pricing fluctuates. Two fields address this: `qte_valid_until` (the operational anchor — a daily query on this date produces the action list) and `qte_expiry_action` (a controlled vocabulary: *Renewed*, *Cancelled*, *Expired-No-Action*, *Superseded*). A structured expiry action makes the question "how many quotes expired with no follow-up in Q2?" answerable with a single WHERE clause.

Tracking the Conversation

`qte_date` is the system-generated creation timestamp — the reference for every pipeline age calculation and expiry measurement. `qte_sent_date` records when the quote was delivered to the customer. `qte_response_date` records when they replied. The gap between sent and response is the customer's decision window — patterns in that window, by customer type or quote value, directly inform follow-up cadence and pipeline forecasting.

TIER II ADDITIONS: `QUOTE` TABLE

Column	Attribute	Notes
<code>qte_date</code>	Quote Date	Timestamp when quote was created; system-generated
<code>qte_status</code>	Quote Status	Draft · Sent · Accepted · Cancelled
<code>qte_est_id</code>	Estimate ID	FK → estimate.est_id; estimate this quote was derived from
<code>qte_parent_id</code>	Parent Quote ID	FK → quote.qte_id; NULL on original; prior version on revisions
<code>qte_version</code>	Quote Version	Integer; 1 = original, increment per revision
<code>qte_prepared_by</code>	Prepared By	FK → user.user_id

qte_approved_by	Approved By	FK → user.user_id; NULL if not yet approved
qte_approved_date	Approval Date	Date approval was granted; NULL if pending
qte_sent_date	Sent Date	Date delivered to the customer
qte_response_date	Response Date	Date customer replied (accepted, rejected, or countered)
qte_valid_until	Valid Until Date	Date the quote is no longer actionable without reissue
qte_expiry_action	Expiry Action	Renewed · Cancelled · Expired-No-Action · Superseded

NEW TABLE: **QUOTE_LINE**

Estimate lines hold internal cost logic. Quote lines hold the customer-facing representation — the same items without exposing cost, margin, or SKU catalog entries.

Column	Attribute	Notes
qte_ln_id	Quote Line ID	Primary key
qte_id	Quote ID	FK → quote.qte_id
est_line_id	Estimate Line ID	FK → estimate_line.est_line_id; connects each quote line to its cost source
qte_ln_description	Line Description	Customer-facing description
qte_ln_qty	Quantity	Quoted quantity
qte_ln_unit_price	Unit Price	Price per unit as presented to the customer
qte_ln_total	Line Total	Calculated: qte_ln_qty × qte_ln_unit_price

Queries That Become Possible

Expired quotes without a closed status — the daily operations query

```
SELECT q.qte_id, q.qte_version, c.cust_name, q.qte_price,
       q.qte_valid_until,
       u.user_first_name || ' ' || u.user_last_name AS prepared_by
FROM quote q
JOIN customer c ON q.cust_id = c.cust_id
JOIN "user" u ON q.qte_prepared_by = u.user_id
WHERE q.qte_valid_until < CURRENT_DATE
      AND q.qte_expiry_action IS NULL
ORDER BY q.qte_valid_until ASC;
```

Each row is a follow-up action — renew, cancel, or log the outcome.

Average revisions before acceptance, by customer type

```
SELECT c.cust_type,
       AVG(rc.version_count) AS avg_revisions_to_acceptance
FROM (
  SELECT q.cust_id,
         COALESCE(q.qte_parent_id, q.qte_id) AS quote_group,
         COUNT(q.qte_id) AS version_count
  FROM quote q
  WHERE q.qte_id IN (SELECT qte_id FROM "order")
  GROUP BY q.cust_id, COALESCE(q.qte_parent_id, q.qte_id)
) rc
JOIN customer c ON rc.cust_id = c.cust_id
GROUP BY c.cust_type ORDER BY avg_revisions_to_acceptance DESC;
```

Quoted vs. estimated price delta by customer tier

```

SELECT c.cust_type, COUNT(*) AS quote_count,
       ROUND(AVG(q.qte_price - e.est_price), 2) AS avg_price_delta,
       ROUND(AVG((q.qte_price - e.est_price) / NULLIF(e.est_price,0) * 100), 2) AS
avg_pct_delta
FROM quote q
JOIN estimate e ON q.qte_est_id = e.est_id
JOIN customer c ON q.cust_id = c.cust_id
WHERE q.qte_version = 1
GROUP BY c.cust_type ORDER BY avg_pct_delta ASC;

```

A negative `avg_price_delta` means quotes are going out below estimate — margin is being given away at the quote stage.

Conversion rate by approver

```

SELECT u.user_first_name || ' ' || u.user_last_name AS approver,
       COUNT(q.qte_id) AS quotes_approved,
       COUNT(o.ord_id) AS quotes_converted,
       ROUND(COUNT(o.ord_id)::decimal / NULLIF(COUNT(q.qte_id),0) * 100, 1) AS
conversion_rate_pct
FROM quote q
JOIN "user" u ON q.qte_approved_by = u.user_id
LEFT JOIN "order" o ON q.qte_id = o.qte_id
WHERE q.qte_approved_by IS NOT NULL
GROUP BY u.user_id, u.user_first_name, u.user_last_name
ORDER BY conversion_rate_pct DESC;

```

Days from quote sent to customer response, by customer tier

```

SELECT c.cust_type,
       COUNT(*) AS responded_quotes,
       ROUND(AVG(q.qte_response_date - q.qte_sent_date), 1) AS avg_days_to_response,
       MIN(q.qte_response_date - q.qte_sent_date) AS fastest_response,
       MAX(q.qte_response_date - q.qte_sent_date) AS slowest_response
FROM quote q
JOIN customer c ON q.cust_id = c.cust_id
WHERE q.qte_sent_date IS NOT NULL
      AND q.qte_response_date IS NOT NULL
GROUP BY c.cust_type ORDER BY avg_days_to_response ASC;

```

If Reseller accounts respond in 3 days but Direct accounts take 12, the follow-up playbook should reflect that difference.

Questions Worth Sitting With

- *In your current process, how many versions does a typical quote go through before it is accepted or rejected? Is that number tracked anywhere today?*
 - *Think about the last time a quote went out and the customer came back with a counteroffer. What changed? Was that change captured anywhere in your data — or did it live only in an email thread?*
 - *Who in your organization has the authority to approve a quote? Is there a formal threshold — price, margin, delivery timeline — that triggers a review, or is the process informal?*
 - *Consider the difference between the quote document your customer receives and the quote record in your system. What information does the document contain that the record does not — and vice versa?*
 - *How many quotes in your current pipeline have passed their stated validity date without a recorded outcome? What would it mean for your pipeline accuracy if you could answer that with a single query?*
 - *Think about a change that happened after a customer accepted a quote — a price correction, a date push, a quantity change. Now that you understand the boundary between revisions and adjustments, where would that change belong in this schema?*
-



CHAPTER 5

Managing the Order

Tier II → III · Order lines, scheduling, partial fulfillment, schema vs. database functions

The First Commitment Record

The quote has been accepted. The customer has said yes. At this moment, the nature of the data changes. Everything up to this point — the request, the estimate, the quote — was *prospective*. It described what might happen. The order is the first record that describes what *will* happen. It is a commitment: your organization has agreed to deliver something specific, at a specific price, by a specific date.

The Tier I order table captured the skeleton of that commitment: a link to the accepted quote, a confirmed cost and price, and an expected delivery date. That is enough to know an order exists. It is not enough to manage one. Consider an order for 200 units of three different products, split across two delivery locations, with one product back-ordered. A single record with a single delivery date and a single price cannot represent that reality. The moment operational complexity enters — and it always does — the header record alone collapses.

One field anchors the entire order in time: `ord_date` — the date the order was confirmed. System-generated at entry, never changed. It is the reference point for every aging, cycle time, and pipeline age calculation the order will ever participate in.

THE DATA GAP

What You Can't Answer Yet

- What specific products or services are included in this order, and in what quantities?
- If an order ships in two deliveries, which line items were in each shipment?
- What delivery date did the customer request, versus what date was operationally promised?
- Which line items are back-ordered or delayed, and by how long?
- What is the fulfillment status of each line item independently?
- How does order composition compare to quote composition — were any items changed at order entry?

Without line-level detail, you cannot route, pick, schedule, or track individual items. Without scheduling fields, you cannot distinguish a customer-requested date from an internal operational target. Without partial fulfillment support, a single delayed line item holds the entire order hostage in your reporting.

CORE CONCEPT

Order Lines: Commitment + Fulfillment State

The `order_line` table mirrors the structure of `estimate_line` and `quote_line` — one row per SKU per order. But it carries something neither predecessor does: **fulfillment state**. An estimate line has a cost. A quote line has a customer-facing price. An order line has a quantity committed, a quantity fulfilled, and a status tracking the gap between them.

Tier II fields are set at order entry — they describe what was ordered. Tier III fields are updated during fulfillment — they describe what happened. This separation makes it possible to query fulfillment state at any point without confusing the original commitment with the operational outcome.

NEW TABLE: `ORDER_LINE`

Column	Attribute	Notes
<code>ord_ln_id</code>	Order Line ID	II Primary key
<code>ord_id</code>	Order ID	II FK → order.ord_id
<code>qte_ln_id</code>	Quote Line ID	II FK → quote_line.qte_ln_id
<code>sku_id</code>	SKU ID	II FK → sku.sku_id
<code>ord_ln_description</code>	Line Description	II Confirmed description of the item or service
<code>ord_ln_qty_ordered</code>	Quantity Ordered	II Committed quantity at order entry
<code>ord_ln_unit_price</code>	Unit Price	II Confirmed price per unit
<code>ord_ln_total</code>	Line Total	II Calculated: qty_ordered × unit_price
<code>ord_ln_qty_fulfilled</code>	Quantity Fulfilled	III Cumulative quantity delivered; updated as deliveries occur
<code>ord_ln_status</code>	Line Status	III Open · Partial · Fulfilled · Cancelled · Back-Ordered
<code>ord_ln_notes</code>	Line Notes	III Substitutions, holds, special handling

CORE CONCEPT

Scheduling: Three Dates, Three Meanings

One of the most common sources of confusion in order management data is the conflation of multiple dates into a single field. “Delivery date” sounds like one thing. In practice, it is at least three — and conflating them makes every schedule-variance query unreliable.

Date	Meaning	Where It Lives
------	---------	----------------

Customer-requested	When the customer wants delivery	request table — already captured at intake via req_id
Promised date	What your org committed to in the quote	ord_promised_date — new at order stage
Operational target	Internal working date based on actual capacity	ord_scheduled_date — set by operations

The customer-requested date is not a new piece of information — it was captured in the `request` table. The order already carries `req_id` as a foreign key, so the requested date is retrieved by joining, not by duplicating. The two dates that are new at the order stage are the promised date and the scheduled date, because they did not exist until the quote was accepted and operations engaged.

Note: `ord_exp_date` from Tier I is superseded by these two fields in Tier II. The three-date view is assembled by joining all three source tables in a single query.

Partial Fulfillment

Not every order ships in a single delivery. Partial fulfillment is managed at two levels. **At the line level:** `ord_ln_qty_fulfilled` tracks cumulative quantity delivered per line; `ord_ln_status` reflects whether a line is open, partially fulfilled, or complete — a line is not Fulfilled until `qty_fulfilled` equals `qty_ordered`. **At the order level:** the order header reflects the aggregate state of its lines — Open if no lines shipped, Partial if some are fulfilled, Fulfilled only when every line is complete.

Schema vs. Database Functions

The schema defines *what* data exists and *how it is structured*. It does not define *how that data is maintained*. Fields marked as “updated as deliveries occur” or “calculated” are candidates for automation in the database layer — **triggers** that execute automatically in response to data events, and **computed fields** calculated on write rather than stored manually.

A trigger watching `order_line` can update the parent `order` automatically: when the first line changes to Partial or Fulfilled, set `ord_status = 'Partial'`; when all lines equal Fulfilled, set `ord_status = 'Fulfilled'` and stamp `ord_fulfilled_date`. Similarly, `ord_ln_total` — defined as `qty × unit price` — is a computed field that guarantees accuracy without depending on human consistency.

This textbook defines the schema. The database implementation — triggers, functions, constraints, computed columns — is built in Phase II. The schema is the specification; the database is the enforcement.

TIER II → III ADDITIONS: ORDER TABLE

Column	Attribute	Notes
ord_date	Order Date	II Timestamp when order was confirmed; system-generated at entry
ord_promised_date	Promised Delivery Date	II Date committed in the accepted quote; sourced from <code>qte_shipment_date</code>
ord_scheduled_date	Scheduled Delivery Date	II Internal operational target; set by operations
ord_status	Order Status	III Open · Partial · Fulfilled · Cancelled · On Hold; maintained by trigger in Phase II
ord_fulfilled_date	Fulfilled Date	

III Date the final line was fulfilled; NULL until complete;
stamped by trigger in Phase II

ord_exp_date from Tier I is superseded. The customer-requested date is accessed via *req_id* → *request* and does not require a separate column on the order.

Queries That Become Possible

Open orders with line-level fulfillment status — the daily operations view

```
SELECT o.ord_id, c.cust_name, o.ord_promised_date, o.ord_scheduled_date,
       o.ord_status,
       COUNT(ol.ord_ln_id) AS total_lines,
       COUNT(CASE WHEN ol.ord_ln_status = 'Fulfilled' THEN 1 END) AS fulfilled_lines,
       COUNT(CASE WHEN ol.ord_ln_status = 'Back-Ordered' THEN 1 END) AS backordered_lines
FROM "order" o
JOIN customer c ON o.cust_id = c.cust_id
JOIN order_line ol ON o.ord_id = ol.ord_id
WHERE o.ord_status IN ('Open', 'Partial')
GROUP BY o.ord_id, c.cust_name, o.ord_promised_date, o.ord_scheduled_date,
         o.ord_status
ORDER BY o.ord_promised_date ASC;
```

Three-date view: request, promise, schedule

```
SELECT o.ord_id, c.cust_name,
       r.req_delivery_date AS customer_requested_date,
       o.ord_promised_date,
       o.ord_scheduled_date,
       o.ord_promised_date - r.req_delivery_date AS promise_gap_days,
       o.ord_scheduled_date - o.ord_promised_date AS schedule_slip_days
FROM "order" o
JOIN customer c ON o.cust_id = c.cust_id
JOIN request r ON o.req_id = r.req_id
WHERE r.req_delivery_date IS NOT NULL
      AND o.ord_promised_date IS NOT NULL
      AND o.ord_scheduled_date IS NOT NULL
ORDER BY promise_gap_days DESC;
```

Demonstrates how the FK chain (ord_id → req_id → request) makes the requested date accessible without storing it redundantly on the order.

Partial fulfillment rate by SKU

```
SELECT s.sku_code, s.sku_name,
       COUNT(ol.ord_ln_id) AS total_order_lines,
       COUNT(CASE WHEN ol.ord_ln_status = 'Back-Ordered' THEN 1 END) AS backordered_lines,
       ROUND(
         COUNT(CASE WHEN ol.ord_ln_status IN ('Partial', 'Back-Ordered') THEN 1
END)::decimal
         / NULLIF(COUNT(ol.ord_ln_id), 0) * 100, 1
       ) AS incomplete_rate_pct
FROM order_line ol
JOIN sku s ON ol.sku_id = s.sku_id
GROUP BY s.sku_id, s.sku_code, s.sku_name
ORDER BY incomplete_rate_pct DESC;
```

Order value variance: quoted price vs. confirmed order price

```
SELECT o.ord_id, c.cust_name,
       q.qte_price AS quoted_price, o.ord_price AS order_price,
       o.ord_price - q.qte_price AS price_delta,
       ROUND((o.ord_price - q.qte_price) / NULLIF(q.qte_price, 0) * 100, 2) AS delta_pct
FROM "order" o
JOIN quote q ON o.qte_id = q.qte_id
JOIN customer c ON o.cust_id = c.cust_id
WHERE o.ord_price <> q.qte_price
ORDER BY ABS(o.ord_price - q.qte_price) DESC;
```

A negative delta means a discount was applied at order entry without a quote revision — a margin leak that bypassed the approval workflow.

Questions Worth Sitting With

- *Think about the most complex order your organization has fulfilled in the past year. How many distinct line items did it contain? How many deliveries did it take? Could your current data model reconstruct a complete picture of that order's fulfillment journey?*
 - *The customer's requested delivery date lives in the request — not the order. Does your current system preserve the original customer ask after the order is confirmed? If not, what is lost when that date is overwritten by the promised or scheduled date?*
 - *Consider a product that has been back-ordered more than once in the past six months. Is that pattern visible in your data? What would it take to surface it with a single query?*
 - *If an order ships in three partial deliveries over three weeks, how does your current system reflect the order's status at each stage? At what point does the order look “closed” in your data — and does that match when the customer considers it complete?*
 - *Think about the fields in this chapter described as “updated as deliveries occur” or “calculated.” In your current environment, who is responsible for keeping those values current? How often are they wrong — and what would it take to make their maintenance automatic?*
-
-



CHAPTER 6

Delivering on the Promise

Tier III · Delivery tracking, actual vs. promised dates, carrier data, partial shipments

The First Backward-Looking Record

The order described what your organization committed to. The delivery describes what actually happened. Every date comparison discussed in previous chapters — requested vs. promised, promised vs. scheduled — has been forward-looking. The delivery record is the first backward-looking entry in the journey: it captures reality, not intention.

It is also the first record your customer experiences directly. They do not see your estimate, your internal margin calculations, or your scheduling variance. They see whether the product arrived, in the right quantity, on the date they needed it. That asymmetry — between what you measured internally and what the customer experienced externally — is what makes the delivery table analytically powerful and operationally critical.

The single most valuable calculation in delivery analytics is also the simplest: `del_date - ord_promised_date`. Positive means late. Negative means early. Zero means on time. But this calculation only works if both dates are in the data and linked to the same record. The foreign key `ord_id` on the delivery record is what connects them — transforming every delivery record from a log entry into a performance measurement.

THE DATA GAP

What You Can't Answer with Tier I

- Was this delivery on time relative to the promised date — or the customer's original request?
- Did the delivery fulfill the entire order, or only part of it?
- Which specific order lines were included in this shipment?
- Who carried it, and by what method?
- If there were multiple deliveries against one order, which one completed it?
- Was anything delivered that was not on the order?

A complete delivery record captures four things: **when** (actual delivery date), **what** (which order lines, in what quantities), **how** (carrier, method, tracking reference), and **against what commitment** (the promised date being measured against). The Tier I record had only the first. Tier III adds the rest.

Tier III: Delivery Table

Column	Attribute	Notes
del_type	Delivery Type	III Full · Partial · Final-Partial — whether this shipment closes the order
del_carrier	Carrier Name	III Shipping carrier or internal delivery resource
del_method	Delivery Method	III Ground · Express · Freight · Internal · Courier
del_tracking	Tracking Reference	III Carrier tracking number or internal shipment ID; audit trail field
del_notes	Delivery Notes	III Access issues, substitutions, condition on arrival

CORE CONCEPT

del_type : Knowing Whether the Order Is Done

The most operationally important field in this table answers the question every operations and customer service team asks when a shipment goes out: *does this close the order?*

- Full** — this single delivery fulfills the entire order
 - Partial** — this delivery fulfills some lines but more shipments are expected
 - Final-Partial** — this is the last shipment; it may not have fulfilled every line completely (back-orders cancelled, substitutions accepted), but the order is considered closed
- `Final-Partial` is the structurally important edge case. It closes the order even when fulfillment was incomplete — capturing the real-world reality that not every order ends with 100% of every line shipped.

CORE CONCEPT

New Table: delivery_line

For full delivery tracking — particularly in multi-shipment orders — a `delivery_line` table records which specific order lines were included in each shipment, and in what quantity. This is the Tier III extension of the line-item pattern established in Chapters 3, 4, and 5.

Column	Attribute	Notes
del_ln_id	Delivery Line ID	III Primary key
del_id	Delivery ID	III FK → delivery.del_id

ord_ln_id	Order Line ID	III	FK → order_line.ord_ln_id; which order line this delivery line fulfills
del_ln_qty	Quantity Delivered	III	Units delivered in this shipment for this line
del_ln_notes	Line Notes	III	Substitutions, damages, short-ships specific to this line

When a `delivery_line` record is created, it triggers an update to the corresponding `order_line`: `ord_ln_qty_fulfilled` increases by `del_ln_qty`, and `ord_ln_status` is recalculated. When all order lines reach Fulfilled — or a `Final-Partial` delivery is recorded — the parent order transitions to Fulfilled and `ord_fulfilled_date` is stamped. This is the trigger chain described in Chapter 5, now shown in its operational context.

Multi-Shipment Orders

Partial delivery is not a failure state — it is a normal operational pattern. Each delivery against an order creates a new `delivery` header and corresponding `delivery_line` rows. The order's fulfillment state is the aggregate of all delivery lines against all order lines.

Example — order with three lines (50 + 20 + 10 units):

First delivery: Lines 1 and 2 ship in full. `del_type = Partial`. Order status: Partial.

Second delivery: Line 3 ships. `del_type = Final-Partial` (or Full). Order status: Fulfilled.
`ord_fulfilled_date` stamped.

Without `delivery_line`, the second delivery looks identical to the first — a timestamp against an order ID. With `delivery_line`, you can reconstruct the complete fulfillment sequence, line by line, shipment by shipment.

CONNECTS FORWARD

The Boundary Between Internal and External

The delivery table sits at the boundary between your internal operations and your customer's reality. Every date comparison it enables — actual vs. promised, actual vs. requested — is a measure of the gap between your commitment and the customer's experience.

- **Chapter 7 — Adjustment:** When a delivery falls short, the adjustment is the mechanism for correcting the record and the relationship. The delivery record is the evidence the adjustment references.
- **Chapter 8 — Response:** The response record captures how the customer reacted. The delivery record captures what they reacted to. Together, they form the complete feedback loop.

`del_carrier` and `del_date`, consistently captured, make carrier performance analysis possible — surfacing which carriers are consistently late and where logistics contracts may need renegotiation.

Queries That Become Possible

On-time delivery rate by month

```
SELECT
  DATE_TRUNC('month', d.del_date) AS delivery_month,
  COUNT(d.del_id) AS total_deliveries,
  COUNT(CASE WHEN d.del_date <= o.ord_promised_date THEN 1 END) AS on_time,
  ROUND(
    COUNT(CASE WHEN d.del_date <= o.ord_promised_date THEN 1 END)::decimal
    / NULLIF(COUNT(d.del_id), 0) * 100, 1
  ) AS on_time_pct
FROM delivery d
JOIN "order" o ON d.ord_id = o.ord_id
WHERE o.ord_promised_date IS NOT NULL
GROUP BY DATE_TRUNC('month', d.del_date)
ORDER BY delivery_month ASC;
```

On-time delivery rate by carrier

```
SELECT d.del_carrier, d.del_method,
  COUNT(d.del_id) AS total_deliveries,
  COUNT(CASE WHEN d.del_date <= o.ord_promised_date THEN 1 END) AS on_time,
  ROUND(
    COUNT(CASE WHEN d.del_date <= o.ord_promised_date THEN 1 END)::decimal
    / NULLIF(COUNT(d.del_id), 0) * 100, 1
  ) AS on_time_pct,
  ROUND(AVG(d.del_date - o.ord_promised_date), 1) AS avg_days_variance
FROM delivery d
JOIN "order" o ON d.ord_id = o.ord_id
WHERE d.del_carrier IS NOT NULL AND o.ord_promised_date IS NOT NULL
GROUP BY d.del_carrier, d.del_method
ORDER BY on_time_pct DESC;
```

avg_days_variance is the richer signal — 85% on-time with +12 days average variance is more damaging than 90% on-time with +4 days.

```
SELECT o.ord_id, c.cust_name,  
       r.req_delivery_date AS customer_requested,  
       o.ord_promised_date AS promised,  
       d.del_date AS actual,  
       o.ord_promised_date - r.req_delivery_date AS promise_gap_days,  
       d.del_date - o.ord_promised_date AS delivery_variance_days,  
       d.del_date - r.req_delivery_date AS total_gap_from_request  
FROM delivery d  
JOIN "order" o ON d.ord_id = o.ord_id  
JOIN request r ON o.req_id = r.req_id  
JOIN customer c ON o.cust_id = c.cust_id  
WHERE d.del_type IN ('Full', 'Final-Partial')  
ORDER BY total_gap_from_request DESC;
```

Filters to Full and Final-Partial ensures one row per closed order. total_gap_from_request is the number that matters most to the customer.

Multi-shipment orders: delivery count and completion timeline

```
SELECT o.ord_id, c.cust_name,
       COUNT(d.del_id) AS delivery_count,
       MIN(d.del_date) AS first_delivery,
       MAX(d.del_date) AS final_delivery,
       MAX(d.del_date) - MIN(d.del_date) AS fulfillment_span_days,
       o.ord_promised_date,
       MAX(d.del_date) - o.ord_promised_date AS final_delivery_variance
FROM delivery d
JOIN "order" o ON d.ord_id = o.ord_id
JOIN customer c ON o.cust_id = c.cust_id
GROUP BY o.ord_id, c.cust_name, o.ord_promised_date
HAVING COUNT(d.del_id) > 1
ORDER BY delivery_count DESC, fulfillment_span_days DESC;
```

Line-level fulfillment completeness

```
SELECT ol.ord_ln_id, o.ord_id, c.cust_name, s.sku_code, s.sku_name,
       ol.ord_ln_qty_ordered,
       COALESCE(SUM(dl.del_ln_qty), 0) AS qty_delivered,
       ol.ord_ln_qty_ordered - COALESCE(SUM(dl.del_ln_qty), 0) AS qty_shortfall,
       ol.ord_ln_status
FROM order_line ol
JOIN "order" o ON ol.ord_id = o.ord_id
JOIN customer c ON o.cust_id = c.cust_id
JOIN sku s ON ol.sku_id = s.sku_id
LEFT JOIN delivery_line dl ON ol.ord_ln_id = dl.ord_ln_id
GROUP BY ol.ord_ln_id, o.ord_id, c.cust_name, s.sku_code, s.sku_name,
         ol.ord_ln_qty_ordered, ol.ord_ln_status
HAVING ol.ord_ln_qty_ordered > COALESCE(SUM(dl.del_ln_qty), 0)
ORDER BY qty_shortfall DESC;
```

The shortfall column feeds directly into the adjustment analysis in Chapter 7.

REFLECTION

Questions Worth Sitting With

- *Think about the last delivery complaint you received from a customer. Was the issue a late delivery, a partial delivery, or something else? Is that distinction visible in your current data?*

- *Your organization likely uses more than one carrier. Do you currently have a way to compare on-time performance across carriers by method? What decisions are being made about logistics contracts without that data?*

- *Consider an order that required three deliveries to complete. In your current system, is it possible to reconstruct the sequence — which lines shipped when, in what quantities, and which delivery closed the order?*

- *The `del_type` field distinguishes Full, Partial, and Final-Partial. Think about a recent order that closed without everything being delivered. How was that closure recorded? Where did the shortfall go?*

- *Think about a recent case where a delivery triggered a correction. What data would you need from the delivery record to support that adjustment clearly?*



CHAPTER 7

When Things Change

Tier III · Adjustment types, root cause analysis, financial impact

The Record of Divergence

Every sales journey assumes a clean path: request to estimate to quote to order to delivery. In practice, that path is rarely clean. Prices get renegotiated after the order is placed. Delivery dates slip. Quantities arrive short. Goods arrive damaged. Credits are issued. Cancellations happen. These are not anomalies — they are the normal friction of doing business, and they occur at every organization, at every scale, in every industry.

The question is not whether changes happen. The question is whether they are captured. When a price is corrected via email and never logged, that correction is invisible to analysis. When a delivery date changes for the fourth time and each change overwrites the last, the pattern is gone — only the final value survives. When a credit is issued but not linked to the delivery or order it was correcting, the financial impact floats in a ledger with no operational context.

The `adjustment` table exists to make these changes visible, structured, and analyzable. It is the record of how reality diverged from the agreement — and what was done about it. The boundary between a quote revision and an adjustment is the order: a quote revision handles changes made *before* the customer accepts; an adjustment handles changes made *after* acceptance.

THE DATA GAP

What You Can't Answer with Tier I

- Who requested this adjustment — customer-initiated or internally identified?
- Who approved it, and when? Is it pending, approved, or already applied?
- Did this adjustment result in a credit to the customer?
- How many adjustments on this order were caused by the same root issue?
- What is the total financial impact of adjustments caused by late delivery this quarter?
- Which product categories generate the most price adjustments?
- Is there a pattern of adjustments clustering around a specific carrier, estimator, or customer type?

Tier III: Adjustment Table

Column	Attribute	Notes
adj_initiated_by	Initiated By	III FK → user.user_id; who logged or raised the adjustment; NULL if customer-initiated externally
adj_initiated_date	Initiated Date	III When the adjustment was first recorded
adj_approved_by	Approved By	III FK → user.user_id; who authorized the adjustment; NULL if not yet approved
adj_approved_date	Approved Date	III When approval was granted
adj_status	Adjustment Status	III Pending · Approved · Applied · Rejected · Cancelled
adj_root_cause	Root Cause	III Carrier Delay · Estimating Error · Customer Change · Supplier Shortage · Damage in Transit · Pricing Error
adj_credit_issued	Credit Issued	III Boolean: whether a financial credit was issued to the customer
adj_credit_amount	Credit Amount	III Dollar value of any credit issued; NULL if no credit
adj_source	Adjustment Source	III Customer-Initiated · Internal-Identified · Carrier-Claim · System-Generated

CORE CONCEPT

Adjustment Types: What Changed

The `adj_type` field classifies the kind of change. Note that `Substitution` is structurally distinct from `Quantity`: a substitution means something *different* was delivered, not just the wrong amount of the right thing.

adj_type	What It Represents
Price	A change to the agreed price on an order or line
Date	A revision to the committed delivery or completion date
Quantity	A change to the ordered or delivered quantity
Credit	A financial concession not tied to a specific price or quantity change
Cancellation	Full or partial cancellation of an order or line
Substitution	A different product or service delivered in place of the original

adj_status : Making Adjustments Operational

Without a status field, the adjustment table is a historical log. With it, the table becomes a workflow queue. A query filtering for `adj_status = 'Pending'` ordered by `adj_initiated_date` is the daily adjustment queue.

Pending — logged but not yet reviewed or approved

Approved — authorized but not yet applied to the relevant records

Applied — the change has been made (price updated, credit issued, date revised)

Rejected — reviewed and denied; the original terms stand

Cancelled — withdrawn before a decision was made

adj_root_cause : From Events to Patterns

`adj_reason` in Tier I is a free-text field — useful for context, not for aggregation. `adj_root_cause` is a structured classification field with a controlled vocabulary. Where `adj_reason` might say *"carrier picked up late due to weather"*, `adj_root_cause` says `Carrier Delay`. The free-text note preserves the narrative. The structured field enables the query. With `adj_root_cause` consistently populated, a single GROUP BY turns the adjustment log into a root cause analysis: how many adjustments, and how much financial impact, originated from each cause category.

adj_source : Who Is Driving the Changes

The ratio of Customer-Initiated to Internal-Identified adjustments is a quality culture metric. Organizations that catch their own errors before customers do are operating with higher internal standards.

Customer-Initiated — the customer raised the issue; this is reactive

Internal-Identified — your team caught it proactively; this reflects operational discipline

Carrier-Claim — originated from a carrier dispute or insurance process

System-Generated — triggered automatically by a tolerance breach or database rule

The Adjustment as a Financial Record

The `adj_cost` and `adj_price` fields capture the delta — the change in cost and price resulting from the adjustment (signed values: negative = reduction). The total financial impact gives you the **realized order value**: what was actually charged and what it actually cost, after all post-acceptance changes are accounted for. An order that looked like 30% margin at the quote stage may have eroded to 18% by the time all adjustments were applied. Without the adjustment table, that erosion is invisible.

Open adjustment queue — what needs a decision today?

```
SELECT
  a.adj_id, a.adj_type, a.adj_root_cause, a.adj_source,
  c.cust_name, o.ord_id,
  a.adj_price AS price_delta,
  a.adj_initiated_date,
  CURRENT_DATE - a.adj_initiated_date AS days_open,
  u.user_first_name || ' ' || u.user_last_name AS initiated_by
FROM adjustment a
JOIN "order" o ON a.ord_id = o.ord_id
JOIN customer c ON a.cust_id = c.cust_id
LEFT JOIN "user" u ON a.adj_initiated_by = u.user_id
WHERE a.adj_status = 'Pending'
ORDER BY a.adj_initiated_date ASC;
```

days_open surfaces how long each item has been sitting without a decision.

Root cause analysis: frequency and financial impact

```
SELECT
  a.adj_root_cause, a.adj_type,
  COUNT(a.adj_id) AS adjustment_count,
  ROUND(SUM(ABS(a.adj_price)), 2) AS total_price_impact,
  ROUND(AVG(ABS(a.adj_price)), 2) AS avg_price_impact,
  COUNT(CASE WHEN a.adj_credit_issued THEN 1 END) AS credits_issued,
  ROUND(SUM(COALESCE(a.adj_credit_amount, 0)), 2) AS total_credits
FROM adjustment a
WHERE a.adj_status = 'Applied'
```

```
GROUP BY a.adj_root_cause, a.adj_type  
ORDER BY total_price_impact DESC;
```

Frequency vs. total impact distinguishes process friction from structural failures.

Margin erosion: quote to final settlement

```
SELECT o.ord_id, c.cust_name, c.cust_type,
       q.qte_price AS quoted_price,
       o.ord_price AS order_price,
       COALESCE(SUM(a.adj_price), 0) AS total_price_adj,
       COALESCE(SUM(a.adj_cost), 0) AS total_cost_adj,
       o.ord_price + COALESCE(SUM(a.adj_price), 0) AS realized_price,
       o.ord_cost + COALESCE(SUM(a.adj_cost), 0) AS realized_cost,
       ROUND(
         (o.ord_price + COALESCE(SUM(a.adj_price), 0) - o.ord_cost -
          COALESCE(SUM(a.adj_cost), 0))
         / NULLIF(o.ord_price + COALESCE(SUM(a.adj_price), 0), 0) * 100
       , 2) AS realized_margin_pct
FROM "order" o
JOIN quote q ON o.qte_id = q.qte_id
JOIN customer c ON o.cust_id = c.cust_id
LEFT JOIN adjustment a ON a.ord_id = o.ord_id AND a.adj_status = 'Applied'
GROUP BY o.ord_id, c.cust_name, c.cust_type, q.qte_price, o.ord_price, o.ord_cost
ORDER BY realized_margin_pct ASC;
```

Orders at the bottom are deals that looked profitable at signing but were not.

Customer-initiated vs. internal-identified by quarter

```
SELECT
  DATE_TRUNC('quarter', a.adj_initiated_date) AS quarter,
  a.adj_source,
  COUNT(a.adj_id) AS adjustment_count,
  ROUND(SUM(ABS(a.adj_price)), 2) AS total_price_impact
FROM adjustment a
WHERE a.adj_status IN ('Approved', 'Applied')
      AND a.adj_source IN ('Customer-Initiated', 'Internal-Identified')
GROUP BY DATE_TRUNC('quarter', a.adj_initiated_date), a.adj_source
ORDER BY quarter ASC, a.adj_source ASC;
```

A healthy trend shows Internal-Identified growing relative to Customer-Initiated.

Delivery-linked adjustments: what late deliveries actually cost

```
SELECT
  d.del_carrier, a.adj_root_cause,
  COUNT(a.adj_id) AS adjustment_count,
  ROUND(SUM(ABS(a.adj_price)), 2) AS total_price_impact,
  ROUND(SUM(COALESCE(a.adj_credit_amount, 0)), 2) AS total_credits_issued
FROM adjustment a
JOIN delivery d ON a.del_id = d.del_id
WHERE a.adj_root_cause IN ('Carrier Delay', 'Damage in Transit')
  AND a.adj_status = 'Applied'
GROUP BY d.del_carrier, a.adj_root_cause
ORDER BY total_price_impact DESC;
```

High credit exposure per late delivery is a different risk profile than frequent-but-cheap lateness.

Questions Worth Sitting With

- *Think about the last time a price was changed after an order was placed. Was that change logged anywhere in your system — or did it happen in an email, a phone call, or a verbal agreement that left no data trail?*
 - *In your current process, what is the most common reason for an adjustment? Is that pattern visible in your data, or is it something you know from experience because you see it repeatedly but cannot prove with a query?*
 - *Consider the distinction between `adj_reason` (free text, narrative) and `adj_root_cause` (structured classification). Who would be responsible for setting the controlled vocabulary for root causes in your organization? What are the five most important categories for your business?*
 - *Think about the ratio of customer-initiated to internally-identified adjustments in your current operation. If you had to estimate it, what would it be? What would it mean if the number was worse than you expected?*
 - *The margin erosion query traces the realized margin from quote to final settlement. If you ran that query against your last year of orders, which customer types or product categories do you suspect have the largest gap between quoted and realized margin?*
-



CHAPTER 8

The Customer's Voice

Tier III · Satisfaction data, NPS, follow-up tracking, closing the loop

The Record That Closes the Loop

Every chapter in this book has described data your organization creates: the request you logged, the estimate you built, the quote you sent, the order you confirmed, the delivery you made, the adjustment you applied. The response record is different. It captures what the customer experienced — their words, their rating, their signal of whether they intend to return.

The response is structurally the last node in the journey, but operationally it is the beginning of the next one. A satisfied customer becomes a new request. A dissatisfied customer who was followed up with and resolved becomes a recoverable relationship. A dissatisfied customer who was never contacted becomes a lost account — often without you knowing why. The response record determines which of those three outcomes is visible and which is invisible.

Most organizations collect some form of customer feedback. Very few connect it back to the specific delivery, order, or adjustment that drove it. Without that connection, feedback is a feeling. With it, feedback is evidence — and evidence can be acted on, trended, and used to hold operational decisions accountable.

THE DATA GAP

What You Can't Answer with Tier I

- What was the customer's satisfaction score for this specific delivery?
- Was a follow-up conducted after a negative response — and did it resolve the issue?
- What is the NPS score by customer segment, product category, or delivery performance band?
- Which customers have not responded at all — and how long have they been silent?
- Did the adjustment we issued actually improve the customer's reported satisfaction?
- Which response channels (email, phone, portal) have the highest response rates?
- What is the relationship between late delivery and satisfaction score in our data?

Tier III: Response Table

Column	Attribute	Notes
resp_channel	Response Channel	III Email · Phone · Portal · In-Person · Survey-Link — how the response was received
resp_date	Response Date	III When the customer submitted or delivered the response
resp_score	Satisfaction Score	III Numeric; scale defined by organization (e.g. 1–5 or 1–10 CSAT)
resp_nps	NPS Score	III 0–10 Net Promoter Score; NULL if not collected on this response
resp_sentiment	Sentiment Classification	III Positive · Neutral · Negative — structured classification derived from resp_text or manual review
resp_text	Response Text	III Free-text comment from the customer; nullable
resp_followup_required	Follow-Up Required	III Boolean; flags responses that require action — driven by score threshold or manual flag
resp_followup_date	Follow-Up Date	III When follow-up was completed; NULL if not yet actioned
resp_followup_by	Follow-Up By	III FK → user.user_id; who conducted the follow-up
resp_resolved	Issue Resolved	III Boolean; outcome of follow-up; NULL if no follow-up required or not yet completed

CORE CONCEPT

Structuring Feedback as Data

The `resp_text` field preserves the customer's words. The `resp_sentiment` field makes those words queryable. The two fields serve different purposes and should both be kept: free text captures nuance that classification flattens; structured classification enables the aggregation that free text prevents.

- `resp_score` — operational satisfaction; measured per transaction
- `resp_nps` — relationship loyalty; measures likelihood to recommend; typically collected less frequently
- `resp_sentiment` — qualitative signal; derived from text or manual review; bridges narrative to analytics

A response table that captures all three gives you three distinct lenses: *how did this specific delivery go* (score), *how loyal is this customer overall* (NPS), and *what tone are they using when they talk about us* (sentiment).

NPS: Promoters, Passives, Detractors

NPS (Net Promoter Score) classifies customers by their `resp_nps` value into three groups. The score itself is calculated as: *% Promoters – % Detractors*. It ranges from –100 to +100. The classification is what makes it actionable by segment.

Promoters (9–10) — loyal advocates; likely to refer new customers

Passives (7–8) — satisfied but not enthusiastic; vulnerable to competitive offers

Detractors (0–6) — unhappy; at risk of churn and negative word-of-mouth

NPS is most valuable when trended over time and segmented by customer type or delivery performance — not as a single aggregate number.

The Follow-Up Queue

The `resp_followup_required` flag transforms the response table from a historical record into an active queue — similar to how `adj_status = 'Pending'` creates the adjustment queue in Chapter 7. When a response below a defined score threshold is received, `resp_followup_required` is set to TRUE. That flag, combined with a NULL `resp_followup_date`, identifies every open follow-up: responses that need action but have not yet received it.

`resp_followup_required = TRUE, resp_followup_date IS NULL` — open; action needed

`resp_followup_required = TRUE, resp_followup_date IS NOT NULL, resp_resolved = TRUE` — closed successfully

`resp_followup_required = TRUE, resp_followup_date IS NOT NULL, resp_resolved = FALSE` — closed but unresolved; escalation candidate

Organizations that consistently close their follow-up queue have measurably higher retention rates than those that treat negative feedback as a log entry.

Queries That Become Possible

Satisfaction score by delivery performance band

```
SELECT
  CASE
    WHEN d.del_date < o.ord_promised_date THEN 'Early'
    WHEN d.del_date = o.ord_promised_date THEN 'On Time'
    WHEN d.del_date <= o.ord_promised_date + INTERVAL '3 days' THEN 'Late ≤3d'
    ELSE 'Late >3d'
  END AS delivery_band,
  COUNT(r.resp_id) AS response_count,
  ROUND(AVG(r.resp_score), 2) AS avg_satisfaction,
  ROUND(AVG(r.resp_nps), 2) AS avg_nps
FROM response r
JOIN delivery d ON r.del_id = d.del_id
JOIN "order" o ON d.ord_id = o.ord_id
WHERE r.resp_score IS NOT NULL AND o.ord_promised_date IS NOT NULL
GROUP BY delivery_band
ORDER BY avg_satisfaction DESC;
```

This query quantifies the satisfaction cost of lateness — making the business case for on-time delivery in the customer's own words.

NPS by customer segment and quarter

```
SELECT
  DATE_TRUNC('quarter', r.resp_date) AS quarter,
  c.cust_type,
  COUNT(r.resp_id) AS responses,
  ROUND(
    100.0 * COUNT(CASE WHEN r.resp_nps >= 9 THEN 1 END) / NULLIF(COUNT(r.resp_nps),
0)
    - 100.0 * COUNT(CASE WHEN r.resp_nps <= 6 THEN 1 END) / NULLIF(COUNT(r.resp_nps),
0)
    , 1) AS nps_score
FROM response r
JOIN customer c ON r.cust_id = c.cust_id
```

```
WHERE r.resp_nps IS NOT NULL  
GROUP BY DATE_TRUNC('quarter', r.resp_date), c.cust_type  
ORDER BY quarter ASC, nps_score DESC;
```

Trending NPS by quarter reveals whether relationship loyalty is improving or eroding — segmented by the customer types that matter most to your business.

Adjustment impact on satisfaction: did the correction work?

```
SELECT
  c.cust_name, o.ord_id,
  COUNT(a.adj_id) AS adjustments_on_order,
  ROUND(SUM(ABS(a.adj_price)), 2) AS total_adj_value,
  r.resp_score,
  r.resp_nps,
  r.resp_sentiment,
  r.resp_resolved
FROM response r
JOIN delivery d ON r.del_id = d.del_id
JOIN "order" o ON d.ord_id = o.ord_id
JOIN customer c ON r.cust_id = c.cust_id
LEFT JOIN adjustment a ON a.ord_id = o.ord_id AND a.adj_status = 'Applied'
WHERE a.adj_id IS NOT NULL
GROUP BY c.cust_name, o.ord_id, r.resp_score, r.resp_nps, r.resp_sentiment,
r.resp_resolved
ORDER BY adjustments_on_order DESC, r.resp_score ASC;
```

Orders with high adjustment values but strong satisfaction scores indicate the correction process is working. Low scores despite corrections signal a deeper relationship issue.

Open follow-up queue — negative responses awaiting action

```
SELECT
  r.resp_id, c.cust_name, c.cust_type,
  r.resp_date, r.resp_score, r.resp_nps,
  r.resp_sentiment, r.resp_text,
  CURRENT_DATE - r.resp_date AS days_open
FROM response r
JOIN customer c ON r.cust_id = c.cust_id
WHERE r.resp_followup_required = TRUE
  AND r.resp_followup_date IS NULL
ORDER BY r.resp_score ASC, r.resp_date ASC;
```

Sort by score first so the most critical voices are always at the top of the queue. `days_open` surfaces how long customers have been waiting to hear back.

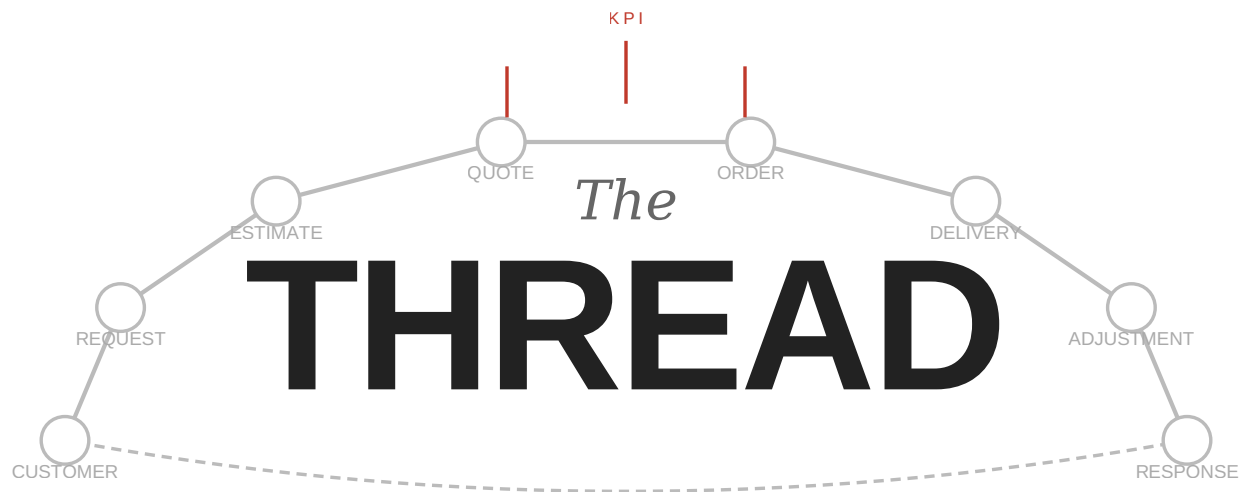
Silent customers — no response in the last 90 days

```
SELECT
  c.cust_id, c.cust_name, c.cust_type,
  MAX(r.resp_date) AS last_response_date,
  CURRENT_DATE - MAX(r.resp_date) AS days_since_last_response,
  COUNT(o.ord_id) AS orders_in_window
FROM customer c
JOIN "order" o ON c.cust_id = o.cust_id
  AND o.ord_exp_date >= CURRENT_DATE - INTERVAL '90 days'
LEFT JOIN response r ON c.cust_id = r.cust_id
GROUP BY c.cust_id, c.cust_name, c.cust_type
HAVING MAX(r.resp_date) < CURRENT_DATE - INTERVAL '90 days'
  OR MAX(r.resp_date) IS NULL
ORDER BY days_since_last_response DESC;
```

Silent customers are not necessarily satisfied — they may have disengaged. Orders in window confirms they have been active recently, making the silence more significant.

Questions Worth Sitting With

- *Think about the last time a customer expressed dissatisfaction. Was that feedback connected to a specific delivery, order, or adjustment in your data — or did it exist only in an email thread, a CRM note, or someone's memory?*
 - *Consider the distinction between a customer who complains and a customer who goes silent. Which is more dangerous to your business? Which is more visible in your current data?*
 - *If you ran the follow-up queue query against your current data, how many open negative responses would surface? What would the average days_open be? Is there a threshold your organization has agreed on — and is it being met?*
 - *Think about your highest-value customers. When did you last receive a response from each of them? Could any of them be silent right now, with active orders, and no feedback in the system?*
 - *The adjustment-to-satisfaction query asks whether your corrections are actually working. Think of a recent case where a significant credit or adjustment was issued. Did you measure whether the customer's satisfaction improved afterward — or was issuing the credit considered the end of the process?*
-
-



CHAPTER 9

Measuring Performance

Tier IV · KPI design, period tables, dashboard logic, period-over-period analysis

The Problem with “How Are We Doing?”

Every business asks it. Most cannot answer it precisely. The schema built across the previous eight chapters contains every transaction in the customer journey — but a row count is not a metric, and a metric is not a KPI. This chapter is about the gap between having data and having a number that means something.

The transactional tables in Tiers I–III are optimized for recording reality accurately. They are not optimized for answering the question “how are we doing this quarter compared to last?” reliably, quickly, or consistently across teams. Tier IV adds three tables that bridge that gap: the period table, the KPI definition table, and the KPI snapshot table. Together they turn a transactional schema into a measurement system.

CORE CONCEPT

KPI Design: Four Requirements

A well-designed KPI must satisfy all four of the following. If any one is missing, the number is a data point — not a KPI.

Requirement	What It Means
Specific question	<i>The metric answers exactly one question, stated in plain language</i>
Defined calculation	<i>Numerator and denominator are written down, not assumed</i>
Time window	<i>The period is bounded and consistently applied</i>
Baseline	<i>There is a prior period or target to compare against</i>

⚠ WATCH OUT — THE DENOMINATOR PROBLEM

The most common KPI failure is an undefined denominator. “On-time delivery rate” sounds simple until you ask: on-time *out of what*? All deliveries? Only completed ones? Only those with a promised date? Does a delivery with two shipments count once or twice?

Every denominator must be written down. If two people in the same meeting would calculate the same metric differently, the KPI is not defined.

The Six Baseline Metrics — Precisely Defined

The Introduction established six baseline questions any Tier I schema should answer. With the full schema now in place, each can be defined precisely — numerator, denominator, and the boundary conditions that make each definition unambiguous.

Metric	Numerator	Denominator	Key Boundary
On-Time Delivery Rate	Deliveries where <code>del_date</code> ≤ <code>ord_promised_date</code> , <code>status</code> = <code>Delivered</code> , <code>type</code> ≠ <code>Internal-Transfer</code>	All delivered records (same filters) in the period	Partial deliveries tracked separately — partial on-time ≠ full on-time
Realized Margin by Segment	Order revenue + applied price adjustments	Order cost + applied cost adjustments	Only <code>adj_status</code> = <code>'Applied'</code> ; pending adjustments are obligations, not realized
Quote Conversion Rate	<code>qte_id</code> values with a corresponding <code>ord_id</code> within 90 days of <code>qte_sent_date</code>	Quotes where <code>qte_status</code> ≠ <code>'Draft'</code> in the period	90-day window must be standardized; attribution follows the quote date, not the order date
New Customer Rate	Customers whose earliest <code>ord_date</code> falls in the current period	Distinct customers with any order in the period	“New” is measured against full order history, not just current year
Avg Order Cycle Time	SUM of <code>MAX(del_date) - ord_date</code> for fully fulfilled orders	COUNT of fully fulfilled orders in the period	Only orders where all lines = Fulfilled; partials understate true cycle time
Customer Response Rate	<code>del_id</code> values with at least one linked <code>resp_id</code>	All delivered <code>del_id</code> values in the period	Low rate may mean the follow-up process is broken, not that customers are satisfied

Tier IV: The Period Table

Calculating metrics from the transactional schema on demand is expensive and fragile. Tier IV adds three tables that make reporting fast, consistent, and auditable. The most foundational is the period table — deceptively simple, deeply leveraged.

Column	Attribute	Notes
<code>per_id</code>	Period ID	IV Primary key
<code>per_name</code>	Period Name	IV e.g., '2025-Q3', '2025-10', '2025-Annual'
<code>per_type</code>	Period Type	IV Monthly · Quarterly · Annual · Custom
<code>per_start_date</code>	Start Date	IV Inclusive
<code>per_end_date</code>	End Date	IV Inclusive
<code>per_prior_id</code>	Prior Period ID	IV FK → period.per_id (self-reference); enables period-over-period joins in one line
<code>per_is_complete</code>	Is Complete	IV Boolean; incomplete periods must be flagged in dashboards — never silently excluded

Why this matters more than it appears: Without a period table, date boundaries are scattered across hundreds of queries, BI tool fields, and spreadsheet formulas — each slightly different. The cost stays invisible until someone asks why the sales report shows Q3 revenue as \$4.2M and the finance report shows \$3.9M. The answer is almost always a boundary problem. A period table eliminates that class of error entirely. The boundary is defined once, in one place, by one owner.

The self-reference (`per_prior_id`): Prior-period comparison is always one join away — no hardcoded dates, no special-casing for year boundaries, no January vs. December arithmetic. The pattern is the same whether you are comparing months, quarters, fiscal years, or custom campaign windows.

Tier IV: KPI Definition & Snapshot Tables

The `kpi_definition` table solves the denominator problem structurally — by making the definition of every metric a row in the database, not tribal knowledge. The `kpi_snapshot` table freezes calculated values at a point in time so dashboards don't drift when underlying data is updated mid-period.

Column	Attribute	Notes
<code>kpi_id</code>	KPI ID	IV Primary key
<code>kpi_name</code>	KPI Name	IV Short display name
<code>kpi_question</code>	Business Question	IV Plain-language question the KPI answers
<code>kpi_numerator_desc</code>	Numerator Description	IV Written definition; required
<code>kpi_denominator_desc</code>	Denominator Description	IV Written definition; required — never implied
<code>kpi_unit</code>	Unit	IV %, count, days, currency
<code>kpi_owner</code>	Owner	IV Person responsible for the definition; definitions without owners drift
<code>kpi_last_reviewed</code>	Last Reviewed	IV Date the definition was last confirmed; surfaces stale metrics

Column	Attribute	Notes
<code>ks_id</code>	Snapshot ID	IV Primary key
<code>ks_kpi_id</code>	KPI Definition ID	IV FK → <code>kpi_definition</code>
<code>ks_per_id</code>	Period ID	IV FK → <code>period</code>
<code>ks_segment</code>	Segment	IV NULL = all customers; populated for segment-level rows
<code>ks_numerator</code>	Numerator	IV Raw count or sum — preserves audit trail
<code>ks_denominator</code>	Denominator	IV Required; never NULL
<code>ks_value</code>	Calculated Value	IV Numerator / Denominator as defined
<code>ks_calculated_at</code>	Calculated At	IV Timestamp of last calculation
<code>ks_calculated_by</code>	Calculated By	IV Process or user that ran the snapshot

Queries That Become Possible

Open order count with prior period comparison

```
SELECT
  p.per_name AS period,
  ks.ks_value AS open_orders,
  ks_prior.ks_value AS prior_open_orders,
  ROUND((ks.ks_value - ks_prior.ks_value) /
  NULLIF(ks_prior.ks_value, 0) * 100, 1) AS pct_change
FROM kpi_snapshot ks
JOIN period p ON ks.ks_per_id = p.per_id
JOIN kpi_definition kd ON ks.ks_kpi_id = kd.kpi_id
LEFT JOIN period p_prior ON p.per_prior_id = p_prior.per_id
LEFT JOIN kpi_snapshot ks_prior
  ON ks_prior.ks_per_id = p_prior.per_id
  AND ks_prior.ks_kpi_id = ks.ks_kpi_id
  AND ks_prior.ks_segment IS NULL
WHERE kd.kpi_name = 'Open Order Count'
  AND ks.ks_segment IS NULL
  AND p.per_type = 'Monthly'
ORDER BY p.per_start_date DESC;
```

The `per_prior_id` self-join replaces all hardcoded date arithmetic. The same pattern works for any metric, any period type.

On-time delivery rate — rolling 13 months via period table

```
SELECT
  p.per_name,
  COUNT(CASE WHEN d.del_date <= o.ord_promised_date THEN 1 END) AS
on_time,
  COUNT(*) AS total_delivered,
  ROUND(
    COUNT(CASE WHEN d.del_date <= o.ord_promised_date THEN 1 END) *
    100.0
    / NULLIF(COUNT(*), 0), 1
  ) AS on_time_pct
FROM delivery d
JOIN "order" o ON d.ord_id = o.ord_id
JOIN period p ON d.del_date BETWEEN p.per_start_date AND
```

```
p.per_end_date  
WHERE d.del_status = 'Delivered'  
      AND d.del_type != 'Internal-Transfer'  
      AND p.per_type = 'Monthly'  
      AND p.per_start_date >= CURRENT_DATE - INTERVAL '13 months'  
GROUP BY p.per_id, p.per_name, p.per_start_date  
ORDER BY p.per_start_date;
```

Period table join replaces hardcoded date ranges. Adding a carrier or segment breakdown requires only one additional GROUP BY column.

```
SELECT
  kd.kpi_name, kd.kpi_unit,
  p.per_name AS current_period,
  ks.ks_value AS current_value,
  p_prior.per_name AS prior_period,
  ks_prior.ks_value AS prior_value,
  ROUND((ks.ks_value - ks_prior.ks_value) /
  NULLIF(ks_prior.ks_value, 0) * 100, 1) AS pct_change,
  p.per_is_complete AS period_complete
FROM kpi_snapshot ks
JOIN kpi_definition kd ON ks.ks_kpi_id = kd.kpi_id
JOIN period p ON ks.ks_per_id = p.per_id
LEFT JOIN period p_prior ON p.per_prior_id = p_prior.per_id
LEFT JOIN kpi_snapshot ks_prior
  ON ks_prior.ks_per_id = p_prior.per_id
  AND ks_prior.ks_kpi_id = ks.ks_kpi_id
  AND ks_prior.ks_segment IS NULL
WHERE ks.ks_segment IS NULL
  AND p.per_type = 'Monthly'
  AND p.per_end_date = (SELECT MAX(per_end_date) FROM period WHERE
per_type = 'Monthly')
ORDER BY kd.kpi_name;
```

`per_is_complete` surfaces in the result so the presentation layer can flag incomplete periods — rather than silently comparing month-to-date to a full prior month.

CORE CONCEPT

Three Dashboard Rules & Three Period-Over-Period Errors

Dashboard Rules:

1. Every number needs a visible comparison — prior period, target, or both. A standalone metric has no meaning.
2. Incomplete periods must be flagged. Month-to-date vs. full prior month will always look worse. Use `per_is_complete`.
3. Segment before you summarize. An 87% on-time rate may hide one tier at 62% and another at 97%.

⚠️ PERIOD-OVER-PERIOD ERRORS

1. **Comparing incomplete to complete periods** — 19 days vs. 31 days almost always shows a false decline.
2. **Shifting segment boundaries** — if “enterprise” is redefined mid-year, the comparison is not the same population.
3. **Ignoring mix shifts** — overall on-time rate can improve while carrier performance degrades if volume shifts toward shorter routes. Summary metrics hide this; segment metrics surface it.

REFLECTION

Questions Worth Sitting With

- *Pick any metric your organization reports regularly. Write down its numerator and denominator in one sentence each. Is there ambiguity in either? Would two colleagues write the same thing?*
 - *Does your organization have a written definition of “new customer”? Does it distinguish a returning customer from a new one against full history — or only recent history?*
 - *Does your current reporting system flag incomplete periods, or are month-to-date figures routinely compared to full prior months without annotation?*
 - *Name one metric in your organization that shifted favorably while the business outcome it was meant to represent stayed flat or got worse. What mix shift or boundary change explains the gap?*
 - *If the six baseline metrics were calculated for your organization tomorrow, which one would produce the most disagreement — and what would that disagreement reveal about how definitions are managed?*
-



CHAPTER 10

Building a Sales Intelligence System

Tier IV · Full schema assembly, system design, automation, and the living schema

WHAT YOU HAVE BUILT

Nine chapters ago, the schema had four columns and one table. It could tell you a customer existed. It could not tell you what they asked for, what you offered them, what you committed to, whether you delivered it on time, what changed along the way, how they felt about it, or whether you made money.

Now it can answer all of those questions — and connect the answers to each other. The table below is the complete Tier I–IV schema as a single readable object.

COMPLETE SCHEMA

Table	First Tier	Purpose
customer	I	Anchor record; every transaction traces back here
request	I	Initial expression of need
estimate	I	Internal cost and price assessment
quote	I	Formal offer to the customer
order	I	Confirmed commitment
delivery	I	Record of what was actually delivered and when
adjustment	I	Any change to a prior record
response	I	Customer reaction after delivery
sku	II	Product and service catalog with cost
user	II	Internal staff; used for assignments, approvals, contacts
contact	II	Customer-side contacts linked to the customer record
estimate_line	II	Line-level detail within an estimate
quote_line	II	Line-level detail within a quote, linked to estimate lines
order_line	II III	Order line detail; fulfillment status updated through Tier III
period	IV	Named, bounded reporting periods with prior-period linkage
kpi_definition	IV	Formal definitions of each KPI, with ownership
kpi_snapshot	IV	Pre-calculated metric values per period, per segment

What a Sales Intelligence System Is

A dashboard is a presentation layer. A sales intelligence system is the infrastructure beneath it: the schema that stores transactions, the definitions that govern how those transactions are measured, the processes that keep both current, and the interfaces that put the right information in front of the right people at the right time.

A report tells you what happened. A system tells you what is happening and flags what needs attention. The difference is automation, alerts, and freshness — and all three depend on how the schema is populated and maintained.

SYSTEM DESIGN

Four Layers

Layer	Purpose	Schema Dependency
1. Data Capture	Write accurate, complete records at the moment of the transaction	Status fields, controlled vocabularies, system-generated timestamps
2. Automation	Trigger status transitions, generate action queues, schedule KPI snapshots	ord_status , del_status , adj_status , per_is_complete , resp_followup_required
3. CRM Integration	Present the schema through a user-facing application; write back through controlled interfaces	All Tier I–III tables (reads); request, quote, adjustment, response (writes)
4. Reporting	Surface KPI snapshots and period-over-period comparisons to decision-makers	kpi_snapshot , period , kpi_definition

⚠ LAYER 1 IS THE MOST COMMON FAILURE POINT

The schema is only as good as the data written into it. **Status fields** should update when the triggering event occurs, not when someone remembers. **Controlled vocabularies** (`adj_type` , `resp_sentiment` , `resp_category`) should be enforced at the application layer — free-text alternatives are a data quality tax paid on every query. **Timestamps** should be system-generated, not user-entered.

Status Transition	Trigger Event	Automated Action
ord_status → In Progress	First order_line linked to a delivery	Update order status
ord_status → Fulfilled	All ord_ln_status = 'Fulfilled'	Update order, set ord_fulfilled_date
del_status → Delivered	Carrier or manual confirmation	Update delivery, calculate del_days_variance
adj_status → Applied	adj_approved_by populated	Update adjustment, trigger credit if adj_credit_issued = TRUE
per_is_complete → TRUE	per_end_date passes	Trigger KPI snapshot calculation for that period

The Living Schema

No schema survives first contact with production unchanged. Three practices separate schemas that remain useful over time from those that accumulate technical debt until they are replaced.

Practice 1 — Changes flow through `kpi_definition`

When a KPI definition changes, record the change in `kpi_definition` before applying it to snapshot calculations. Update `kpi_last_reviewed`, annotate what changed in the description fields. Historical snapshots are preserved under the prior definition. New snapshots are calculated under the new one. Without this, a trend line that appears to show improvement may actually reflect a redefinition.

Practice 2 — New fields are additive, not overwriting

A field no longer populated is deprecated with a note, not deleted — historical records contain it. A table replaced by a more structured version is retained, not dropped. The cost of keeping deprecated fields is low. The cost of losing historical continuity is high. This is the same principle the tier system used throughout this textbook: add, don't replace.

Practice 3 — Definitions are owned, not assumed

Every `kpi_definition` row has a `kpi_owner`. Every period definition is loaded by a named process. Every schema change is attributable to a decision. This is not bureaucracy — it is the difference between a schema that is auditable and one that is mysterious. When a number changes unexpectedly, the question is always: *was this a change in the business, or a change in how we measured it?* A schema with ownership can answer that. One without it cannot.

ACTION QUEUE AUTOMATION

Several fields in the schema exist specifically to generate action queues, not historical records:

Field	What It Generates
<code>qte_valid_until</code>	Quotes expiring without a linked order — salesperson follow-up list

<code>adj_status = 'Pending'</code> <code>+ adj_initiated_date</code>	Aging approval queue — bottleneck signal for management
<code>resp_followup_required</code> <code>= TRUE AND</code> <code>resp_resolved = FALSE</code>	Open follow-up task list — surfaced to <code>resp_followup_owner</code>
<code>del_status</code> not updated within N days of <code>del_scheduled_date</code>	Delivery confirmation gap — either a process failure or an actual delivery failure

Full Customer Intelligence View — Last 12 Months

Assembles order volume, realized margin, on-time rate, adjustment count, and NPS for every active customer in a single query.

```
WITH customer_orders AS (
  SELECT o.ord_cust_id,
         COUNT(DISTINCT o.ord_id) AS total_orders,
         SUM(ol.ord_ln_unit_price * ol.ord_ln_qty_fulfilled) AS gross_revenue,
         SUM(s.sku_cost * ol.ord_ln_qty_fulfilled) AS gross_cost
  FROM "order" o
  JOIN order_line ol ON o.ord_id = ol.ord_id
  JOIN sku s ON ol.sku_id = s.sku_id
  WHERE o.ord_date >= CURRENT_DATE - INTERVAL '12 months'
  GROUP BY o.ord_cust_id
),
adjustment_impact AS (
  SELECT o.ord_cust_id,
         COUNT(DISTINCT a.adj_id) AS total_adjustments,
         SUM(COALESCE(a.adj_price_delta, 0)) AS total_price_adj
  FROM adjustment a
  JOIN "order" o ON a.adj_ord_id = o.ord_id
  WHERE a.adj_status = 'Applied'
         AND o.ord_date >= CURRENT_DATE - INTERVAL '12 months'
  GROUP BY o.ord_cust_id
),
delivery_performance AS (
  SELECT o.ord_cust_id,
         COUNT(d.del_id) AS total_deliveries,
         COUNT(CASE WHEN d.del_actual_date <= d.del_promised_date THEN 1 END) AS on_time
  FROM delivery d JOIN "order" o ON d.del_ord_id = o.ord_id
  WHERE d.del_status = 'Delivered' AND d.del_type != 'Internal-Transfer'
         AND o.ord_date >= CURRENT_DATE - INTERVAL '12 months'
  GROUP BY o.ord_cust_id
),
nps_summary AS (
  SELECT r.cust_id, ROUND(AVG(r.resp_nps), 1) AS avg_nps, COUNT(r.resp_id) AS
response_count
  FROM response r
  WHERE r.resp_nps IS NOT NULL AND r.resp_date >= CURRENT_DATE - INTERVAL '12 months'
  GROUP BY r.cust_id
)
SELECT c.cust_id, c.cust_name, c.cust_segment,
       COALESCE(co.total_orders, 0) AS orders_12m,
```

```

    ROUND(COALESCE(co.gross_revenue,0) + COALESCE(ai.total_price_adj,0), 2) AS
realized_revenue,
    ROUND((COALESCE(co.gross_revenue,0)+COALESCE(ai.total_price_adj,0)-
COALESCE(co.gross_cost,0))
    / NULLIF(COALESCE(co.gross_revenue,0)+COALESCE(ai.total_price_adj,0),0)*100, 1) AS
margin_pct,
    ROUND(COALESCE(dp.on_time,0)*100.0/NULLIF(dp.total_deliveries,0),1) AS on_time_pct,
    COALESCE(ai.total_adjustments,0) AS adjustments_12m,
    ns.avg_nps, ns.response_count
FROM customer c
LEFT JOIN customer_orders co ON c.cust_id = co.ord_cust_id
LEFT JOIN adjustment_impact ai ON c.cust_id = ai.ord_cust_id
LEFT JOIN delivery_performance dp ON c.cust_id = dp.ord_cust_id
LEFT JOIN nps_summary ns ON c.cust_id = ns.cust_id
WHERE co.total_orders > 0
ORDER BY realized_revenue DESC;

```

Five CTEs, one final SELECT. Every signal from the full customer journey assembled into a single customer intelligence row, ordered by realized revenue.

Open action queue — expiring quotes, pending adjustments, open follow-ups in one view

```
SELECT 'Quote Expiring' AS action_type, c.cust_name, q.qte_id AS record_id,
       q.qte_valid_until AS due_date,
       (julianday(q.qte_valid_until) - julianday('now')) AS days_remaining,
       u.user_first_name || ' ' || u.user_last_name AS owner
FROM quote q JOIN customer c ON q.cust_id = c.cust_id
LEFT JOIN "user" u ON q.qte_prepared_by = u.user_id
WHERE q.qte_status NOT IN ('Accepted', 'Cancelled', 'Expired-No-Action', 'Superseded')
      AND q.qte_valid_until <= CURRENT_DATE + INTERVAL '14 days'
UNION ALL
SELECT 'Adjustment Pending Approval', c.cust_name, a.adj_id,
       a.adj_initiated_date + INTERVAL '5 days',
       (julianday('now') - julianday(a.adj_initiated_date)), u.user_first_name || ' ' ||
u.user_last_name
FROM adjustment a JOIN "order" o ON a.adj_ord_id = o.ord_id
JOIN customer c ON o.ord_cust_id = c.cust_id
LEFT JOIN "user" u ON a.adj_approved_by = u.user_id
WHERE a.adj_status = 'Pending'
UNION ALL
SELECT 'Response Follow-Up Due', c.cust_name, r.resp_id,
       r.resp_date, (julianday('now') - julianday(r.resp_date)),
       u.user_first_name || ' ' || u.user_last_name
FROM response r JOIN customer c ON r.cust_id = c.cust_id
LEFT JOIN "user" u ON r.resp_followup_owner = u.user_id
WHERE r.resp_followup_required = TRUE AND r.resp_resolved = FALSE
ORDER BY days_remaining DESC;
```

System health check — data quality and snapshot freshness

```
SELECT 'Orders with NULL status' AS check_name, COUNT(*) AS issue_count FROM "order"
WHERE ord_status IS NULL
UNION ALL
SELECT 'Deliveries with NULL promised date', COUNT(*) FROM delivery
WHERE del_promised_date IS NULL AND del_status != 'Cancelled'
UNION ALL
SELECT 'Applied adjustments with no root cause', COUNT(*) FROM adjustment
WHERE adj_root_cause IS NULL AND adj_status = 'Applied'
UNION ALL
SELECT 'Responses: score but no scale', COUNT(*) FROM response
WHERE resp_score IS NOT NULL AND resp_score_scale IS NULL
UNION ALL
SELECT 'KPI snapshots older than 48h (complete periods)', COUNT(*)
FROM kpi_snapshot ks JOIN period p ON ks.ks_per_id = p.per_id
```

```
WHERE p.per_is_complete = TRUE AND ks.ks_calculated_at < NOW() - INTERVAL '48 hours'  
ORDER BY issue_count DESC;
```

Run this weekly. Any non-zero count is a defined question: a data entry gap, an automation failure, or a snapshot process that didn't run.

Questions Worth Sitting With

- *Of the four system layers — data capture, automation, CRM integration, and reporting — which one is most likely to be the weakest in your organization today? What would break if you audited it honestly?*
 - *Does your organization currently have a single view across expiring quotes, pending adjustments, and open follow-ups? Or are those three lists maintained separately, by different teams, in different tools?*
 - *The journey completeness audit surfaces requests with no estimate, estimates with no quote, orders with no delivery. How many records in your current system would that query return? What does the answer tell you about where the customer journey breaks down most often?*
 - *The living schema practice of preserving deprecated fields rather than deleting them has a cost: a schema that accumulates columns over time. Where does your organization draw that line, and how is the decision made?*
 - *If the system health check query ran against your database tomorrow, how confident are you in what those counts would be? The answer is a measure of how well you know your own data.*
-

END OF TEXTBOOK

You have reached the end of the textbook. The complete Tier I-IV schema is now assembled. The database implementation supporting this model is built in Phase II using Supabase. The CRM application putting this schema to work with live data is available in Phase III.